
FR800x SDK User Guide

Bluetooth Low Energy SOC with SIG Mesh

Version: V1.1

2021.11



Revision

Version	Date	Description
V1.1	2021.11.1	First Edition

Freqchip Confidential

Contents

Contents	I
Figure.....	1
1. Overview.....	3
1.1. Introduction.....	3
1.2. Program Storage	4
1.3. FLASH Address Allocation	4
1.4. Software Framework	5
1.4.1. Boot Process.....	5
1.4.2. Main Cycle.....	8
1.4.3. Sleep and Wake-up.....	9
1.5. Programming	10
1.5.1. PC Serial Port Programming.....	10
1.5.2. J-Link Programming.....	11
2. BLE Protocol Stack.....	17
2.1. Initialization.....	17
2.1.1. Protocol Stack Initialization	17
2.1.2. GAP callback function	18
2.2. Advertising	18
2.2.1. Legacy Advertising	18
2.2.2. Extended Advertising	21
2.2.3. Periodic Advertising	24
2.2.4. Related GAP Events	25
2.3. Scan.....	26
2.3.1. Traditional scanning.....	26
2.3.2. Extended Scan.....	27
2.3.3. Synchronous Scanning.....	28
2.3.4. GAP Events.....	29
2.4. Connection.....	29
2.4.1. Connect To Legacy Advertising	29
2.4.2. Connection Extended Broadcast.....	31
2.4.3. Other.....	31
2.4.4. Related GAP Events	32
2.5. Definition of Profile	32
2.5.1. GATT Roles	32
2.5.2. GATT Profile Hierarchy	33
2.5.3. Attribute Table	35
2.5.4. Basic Process of GATT Client.....	39

2.5.5. Basic Process of GATT Server	51
2.6. Safety Management	60
2.6.1. Pairing	60
2.6.2. Encryption	69
2.6.3. Privacy Management	71
3. OSAL API	75
3.1. Task	75
3.1.1. Create a Task	75
3.1.2. push a Message to a Task	75
3.2. Timer	76
3.3. Memory	77
4. Peripheral Driver	78
4.1. GPIO	78
4.1.1. GPIO Control (Normal Control Unit)	79
4.1.2. GPIO Control (Low Power Control Unit)	81
4.1.3. GPIO Control (New Driving Method)	83
4.2. ADC	85
4.3. DMA	86
4.4. I2C	87
4.5. Other	88
5. System Function	89
5.1. System Clock	89
5.2. System Sleep	89
5.3. Other	89
5.3.1. Program Running Time	89
5.3.2. SDK Compilation Time	89
6. OTA	90
6.1. Firmware Generation	91
6.2. Firmware Writing	92
Contact Information	93

Figure

Figure 1-1 SDK Block Diagram	3
Figure 1-2 Storage Address Allocation	4
Figure 1-3 FLASH Address Allocation	5
Figure 1-4 Programming Tool Waiting for Connection.....	10
Figure 1-5 Programming Tool Connected	11
Figure 1-6 Tools Sub-tab	12
Figure 1-7 Debug Sub-Tab	12
Figure 1-8 Debug Mode Configuration	13
Figure 1-9 Flash download Sub-Tab.....	13
Figure 1-10 Utilities Sub-Tab	13
Figure 1-11 Storage Location of 10FR8000.FLM.....	14
Figure 1-12 Create New Project1	14
Figure 1-13 Create New Project1	15
Figure 1-14 target device selection	15
Figure 1-15 Connection.....	15
Figure 2-1 Enables Legacy Advertising Process.....	19
Figure 2-2 Start Periodic Advertising Process.....	24
Figure 2-3 Legacy Scanning.....	26
Figure 2-4 Periodic Advertising Synchronization Scan Flow	28
Figure 2-5 Connection Establishment Process.....	30
Figure 2-6 Service, Characteristic, Attribute Hierarchy Relationship	33
Figure 2-7 The process of establishing the attribute table.....	39
Figure 2-8 Client Workflow in SDK	40
Figure 2-9 Client write_command Processing Flow	47
Figure 2-10 Client read processing flow	50
Figure 2-11 Server GATT Basic Process	52

Figure 2-12 Server notification Processing Flow	57
Figure 2-13 Server Read Response Processing Flow.....	59
Figure 2-14 BLE Pairing Process	61
Figure 2-15 Host Initiates Pairing Operation	67
Figure 2-16 Slave Initiates Pairing Operation	68
Figure 6-1 Firmware Information Area Data Format	90
Figure 6-2 The basic format of the compiled bin file is as follows.....	91
Figure 6-3 Fields to be added to the information of the firmware to be upgraded.....	91

1. Overview

This document is a guide for the application development of the FR8000 SDK. The FR8000 SDK is a software package running on the FR8000 series chip, including the complete protocol stack of the BLE 5.0, the peripheral driver of the chip, and the operating system abstraction layer (OSAL), etc.

1.1. Introduction

The architecture of the FR8000 SDK is shown in the following figure. The SDK contains a complete BLE 5.0 protocol stack, including the complete controller, host, profile, and SIG Mesh components. The controller and host parts of the Bluetooth protocol stack and the operating system abstraction layer (OSAL) are all provided in the form of libraries, which are presented in gray in the figure. The MCU peripheral drivers and profile, as well as routine code of the application layer, are provided in the form of source code, which are presented in green in the figure.

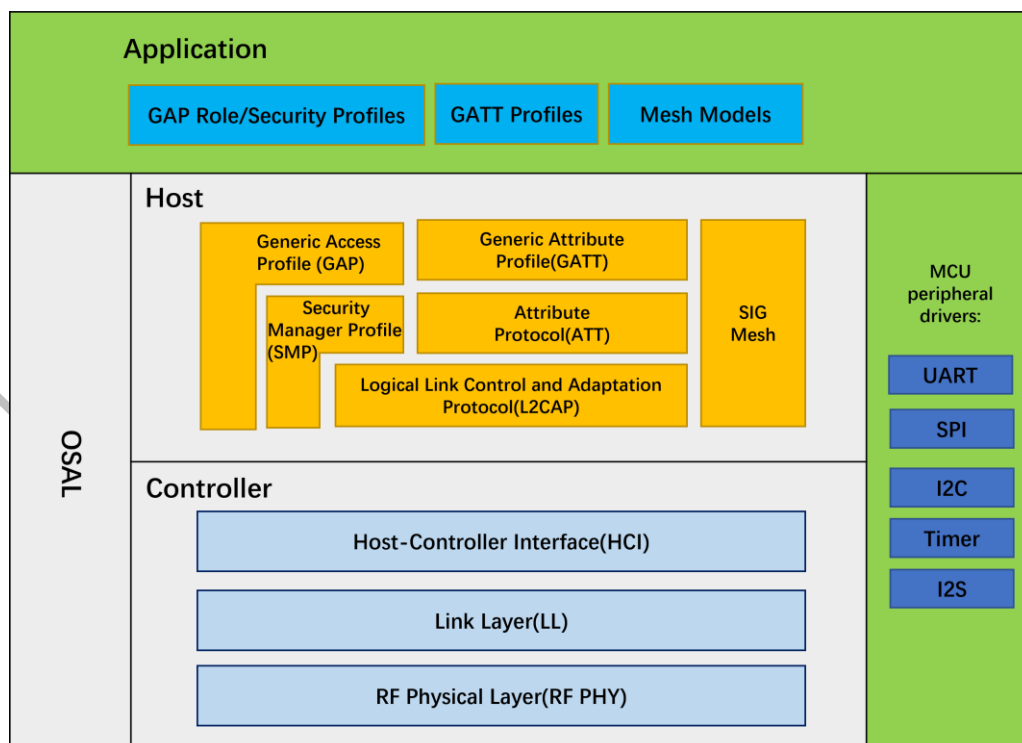


Figure 1-1 SDK Block Diagram

1.2. Program Storage

The storage of FR800x is mainly consists of ROM, FLASH and RAM. The ROM contains a built-in non-modifiable program, which mainly realizes the functions of bootloader and part of the BLE controller protocol stack. FLASH is connected to the AHB bus of the system through cache and supports XIP (running on chip), which is used to store user programs, non-volatile variables, etc. RAM is used to store temporary variables during program operation, key code with requirements for response speed (this part of the code is copied from FLASH when the system starts up).

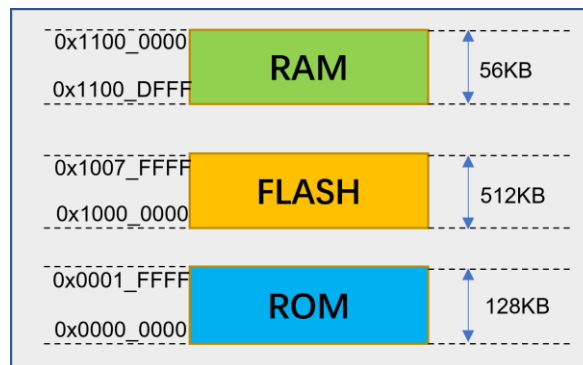


Figure 1-2 Storage Address Allocation

1.3. FLASH Address Allocation

FLASH is mainly used to store user programs and non-volatile data. Due to the user's OTA(Over-The-Air) requirements, the program space adopts a dual-backup method. The non-volatile data includes ECDH public-private key pair, IR Key, Link Key after pairing, etc. required for secure connection, properties of slave devices bound when acting as a master device, user data, etc.

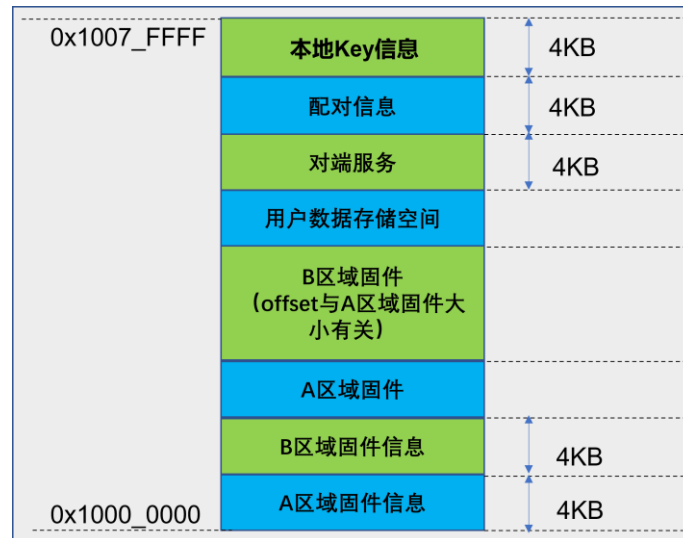


Figure 1-3 FLASH Address Allocation

The program normally runs the firmware in Area A. During OTA upgrade, the new firmware is stored in the "firmware in Area B" and the firmware information is stored in the "firmware information in Area B". After the transmission and verification are completed, the system reboots, and then the bootloader performs firmware upgrade. After a successful upgrade, the firmware information in Area B will be cleared.

1.4. Software Framework

The entire software is divided into two parts: ROM firmware and user program. The user program is stored in FLASH and can run directly on FLASH. After completing various initialization tasks, the system adopts a non-preemptive polling mechanism for scheduling. This section provides a macro-level overview of the program's running process.

1.4.1. Boot Process

The system's default reset vector is in the ROM space. After power - on, the Reset_Handler in the ROM runs first, and then the bootloader in the ROM runs. The bootloader has two main functions: attempting to handshake with the host computer to achieve the programming function (which is used in both the program development phase and the mass - production programming phase); and detecting the content stored in the

FLASH to determine whether the final OTA copy operation needs to be executed. After the bootloader detects a valid firmware program in the FLASH, it will find the entry address of the user program and then jump to the user program. The code executed in this part is as follows. In `app_boot` is the bootloader, and `app_entry` is the obtained entry address of the user program.

```
int main(void)
{
    void (*app_entry)(void);

    memcpy((uint8_t *)&__jump_table, (uint8_t *)app_boot(), sizeof(__jump_table));
    __set_MSP((uint32_t)__jump_table.stack_top_address);
    /* do initialization */
    ...
    app_entry = (void (*)(void))(__jump_table.entry);
    app_entry();
}
```

The entry of the user program is implemented in the library. In the entry program, the initialization of the user program is first carried out, including the initialization of the RW data segment in the user program, clearing of the ZI segment, and copying of key code to the RAM. After remapping the system exception vector address to the starting address of the RAM, the program jumps to the entry point `user_main` of the application - layer code. The code executed in this part is as follows:

```
void app_main(uint8_t index)
{
    uint32_t *dst, *src, *end;
    uint32_t rand_num;

    memset((void *)0x40004000, 0, 8*1024);
    /* init RW section */
    dst = (uint32_t *)&Image$$ER_RW$$RW$$Base;
    src = (uint32_t *)&Load$$ER_RW$$RW$$Base;
    end = (uint32_t *)&Load$$ER_RW$$RW$$Limit;
    for(; (uint32_t)src < (uint32_t)end;)
    {
        *dst++ = *src++;
    }
    /* init ZI section */
    dst = (uint32_t *)&Image$$ER_ZI$$ZI$$Base;
    end = (uint32_t *)&Image$$ER_ZI$$ZI$$Limit;
    for(; dst < end;)
    {
        *dst++ = 0;
    }
    /* copy RAM code from flash to RAM */
}
```

```

    dst = (uint32_t *)&Image$$USER_RE_RAM$$Base;
    src = (uint32_t *)&Load$$USER_RE_RAM$$Base;
    end = (uint32_t *)&Load$$USER_RE_RAM$$Limit;
    for(; (uint32_t)src<(uint32_t)end;)
    {
        *dst++ = *src++;
    }
    /* copy reset handlers from flash to RAM */
    dst = (uint32_t *)&Image$$ER_BOOT$$Base;
    src = (uint32_t *)&Load$$ER_BOOT$$Base;
    end = (uint32_t *)&Load$$ER_BOOT$$Limit;
    for(; (uint32_t)src<(uint32_t)end;)
    {
        *dst++ = *src++;
    }
    /* some other initialization */
    ...
    /* set exception vector offset to RAM space */
    SCB->VTOR = 0x11000000;
    ...
    /* all interrupt priorities are set to 4 except BLE interrupt is set to 2 */
    intc_init();
    /* initial rand seed with internal TRNG */
    void trng_init(void);
    rand_num = trng_init();
    srand(rand_num);
    /* jump to application code */
    user_main();
}

```

The above two parts are implemented in the ROM and the library respectively, and they are invisible to the users. Next, user_main to be run is part of the application program. Inside it, users can configure peripherals and the Bluetooth protocol stack according to the actual project. Here is an example:

```

void user_main(void)
{
    /* initialize log module */
    log_init();
    /* initialize PMU module at the beginning of this program */
    pmu_sub_init();
    /* start lowpower RC clock calibration */
    NVIC_EnableIRQ(PMU_IRQn);
    pmu_calibration_start(PMU_CALI_SEL_RCLFOSC, LP_RC_CALIB_CNT);
    /* set system clock */
    system_set_clock(SYSTEM_CLOCK_SEL);
    jump_table_set_static_keys_store_offset(JUMP_TABLE_STATIC_KEY_OFFSET);
    /* initialize ble stack */
    mac_addr_t mac_addr;
    mac_addr.addr[0] = 0xbd;
    mac_addr.addr[1] = 0xad;
    mac_addr.addr[2] = 0x10;
    mac_addr.addr[3] = 0x11;
    mac_addr.addr[4] = 0x20;
    mac_addr.addr[5] = 0x20;
    gap_address_set(&mac_addr, BLE_ADDR_TYPE_PRIVATE);
}

```

```

    ble_stack_configure(false,
                        BLE_STACK_ENABLE_CONNECTIONS,
                        BLE_STACK_RX_BUFFER_CNT,
                        BLE_STACK_RX_BUFFER_SIZE,
                        BLE_STACK_TX_BUFFER_CNT,
                        BLE_STACK_TX_BUFFER_SIZE,
                        BLE_STACK_ADV_BUFFER_SIZE);

    /* interrupt will be generated and should be handled during ble_stack_init */
    GLOBAL_INT_START();
    ble_stack_init();
    gap_dev_name_set("FR8000", strlen("FR8000"));
    gap_bond_manager_init(BLE_BONDING_INFO_SAVE_ADDR, BLE_REMOTE_SERVICE_SAVE_ADDR, 8, true);
    gap_set_cb_func(proj_ble_gap_evt_func);
    proj_init();
    /* enter main loop */
    main_loop();
}

```

1.4.2. Main Cycle

After completing all the initializations, the system will enter the main loop. The main loop uses a non - preemptive polling scheduling method. Here is an example:

```

__attribute__((section("ram_code"))) void main_loop(void)
{
    while(1) {
        if(ble_stack_schedule_allow()) {
            /*user code should be add here*/

            /* schedule internal stack event */
            ble_stack_schedule();
        }

        GLOBAL_INT_DISABLE();
        switch(ble_stack_sleep_check()) {
            case 2:
                ble_stack_enter_sleep();
                break;
            default:
                break;
        }
        GLOBAL_INT_RESTORE();

        ble_stack_schedule_backward();
    }
}

```

Ble_stack_schedule is the entry point for scheduling. This function will be called when the ble_stack_schedule_allow returns true, which is used to check whether there are new events to be processed. If the user has his own polling code, it can also be added here. It

should be noted that a single execution event cannot be too long (a few dozen ms) to avoid affecting baseband scheduling.

1.4.3. Sleep and Wake-up

The `ble_stack_sleep_check` function mentioned in the previous section is used to check whether the current system state allows the system to enter sleep mode. The check items include whether there are events to be processed, whether there are tasks to be completed within a short period (a few milliseconds), and whether there are Bluetooth scheduling tasks. If it is found that there are no tasks to be processed within a short time, this function will return a result indicating that the system can enter sleep mode. When the system can enter sleep mode, calling `ble_stack_enter_sleep` can make the system enter the sleep state. The system will be woken up when the sleep time is up or an asynchronous event occurs. After the system is woken up, the `ble_stack_schedule_backward` function is used to complete the system restoration tasks.

When the system is in sleep mode, all peripherals and the MCU core are in a power - down state. Therefore, the used peripherals need to be re - initialized after waking up. To prevent GPIO leakage, a fixed state needs to be set for the GPIO during sleep. For this purpose, the program reserves two interfaces: `user_entry_before_sleep_imp` and `user_entry_after_sleep_imp`, which are called before sleep and after wake - up respectively. Users can add necessary programs at these two entry points to implement the actions mentioned above. The following are reference examples of these two functions, which simply implement the enabling and disabling of low - power RC clock calibration:

```
__attribute__((section("ram_code"))) void user_entry_before_sleep_imp(void)
{
    pmu_calibration_stop();
    uart_putc_noint_no_wait(UART0, 's');
}

__attribute__((section("ram_code"))) void user_entry_after_sleep_imp(void)
{
    log_init();
    uart_putc_noint_no_wait(UART0, 'w');

    /* reinit IO and peripherals */
}
```

```
NVIC_EnableIRQ(PMU_IRQn);
pmu_calibration_start(PMU_CALI_SEL_RCLFOSC, LP_RC_CALIB_CNT);

}
```

1.5. Programming

The programming of FR800x is divided into two scenarios: the development phase and the mass - production phase. In the development phase, two programming methods are supported: PC serial port programming and J - Link programming. Among them, J - Link programming supports two programming paths: Keil and J - Flash.

1.5.1. PC Serial Port Programming

Required tools: PC programming software, USB to serial converter.

When the chip is powered on, the internal boot program will attempt to communicate with external tools through the serial port. After a successful handshake, subsequent operations such as programming can be carried out. The specific operations are as follows:

1. Open the serial port programming tool on the PC, select the correct serial port number, import the DAT file (select the bin file to be programmed), and then open the serial port to enter the waiting - for - connection state.



Figure 1-4 Programming Tool Waiting for Connection

- There are two ways for the programming tool to handshake with the chip:

Way 1: Connect the TX of the serial port tool to the PA0 of the chip (RX on the chip side), connect the RX of the serial port tool to the PA1 of the chip (TX on the chip side), then connect the power supply VCC to the VBAT of the chip, and finally connect the ground wire.

Way 2: Connect the TX, RX, VCC, and GND of the serial port tool to the chip, and then press the reset button.

- At this time, after the chip successfully shakes hands with the PC tool, the tool will display "Connected (flash)". Then click "Write all contents" to program the program into the chip.



Figure 1-5 Programming Tool Connected

1.5.2. J-Link Programming

The J-Link pins of the chip are PC6(SWCLK) and PC7(SWO).

1.5.2.1. Keil + jlink Programming

Required tools: keil, J-Link.exe, FR8000.FLM.

The Users should store the file FR8000.FLM in the path ARM\Flash under the Keil installation directory, and then make the following configurations in the Keil project:

1. Select J - Link as the debugging tool.

In the Keil tool menu bar, click on "Flash", then select "Config Flash Tools" from the drop - down menu, as shown in Figure 1 - 6. In the "Debug" sub - tab, select the "J - LINK/J - TRACE Cortex" option, as shown in Figure 1 – 7

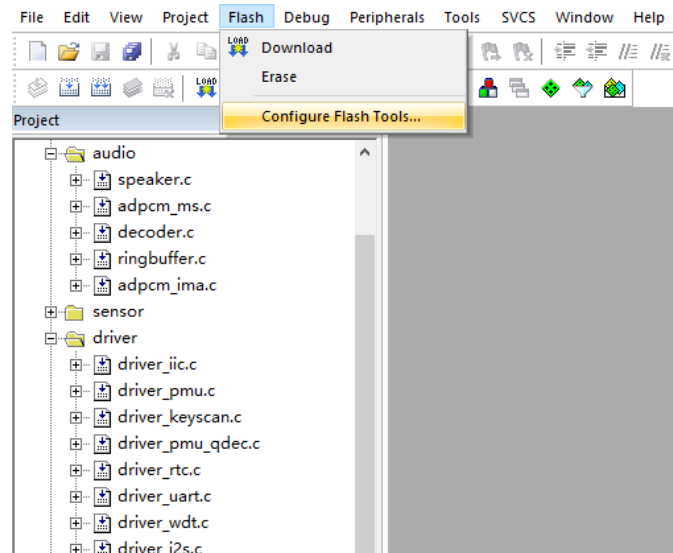


Figure 1-6 Tools Sub-tab

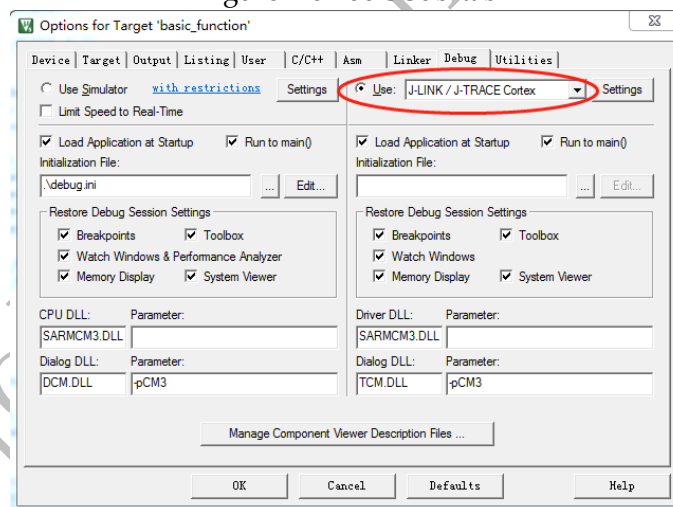


Figure 1-7 Debug Sub-Tab

2. Configure the debugging mode as SW

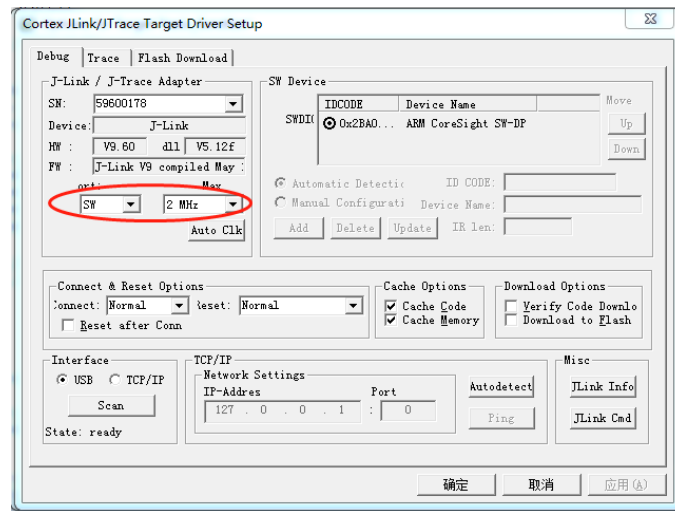


Figure 1-8 Debug Mode Configuration

3. Configure download options in the “flash download” tab

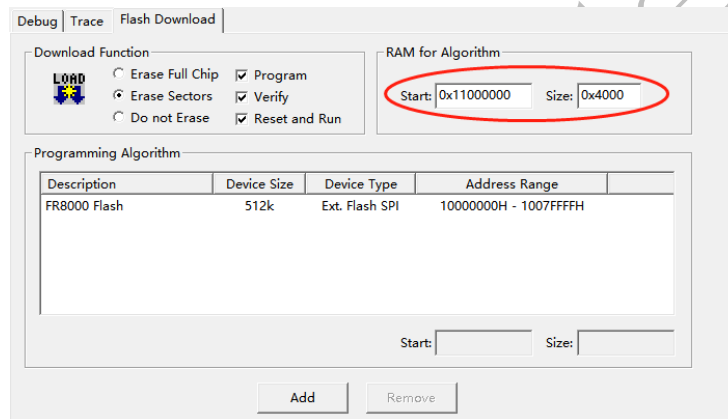


Figure 1-9 Flash download Sub-Tab

4. Configure to use the Debug Driver for flash programming

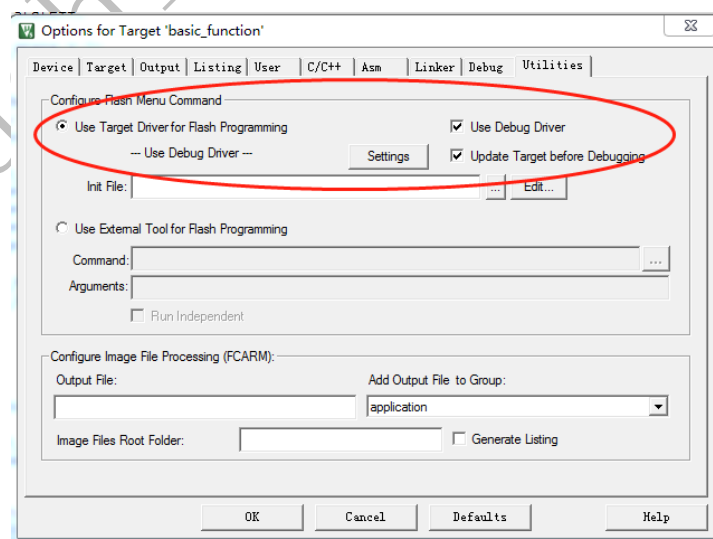


Figure 1-10 Utilities Sub-Tab

With the above configurations, you can debug and program the flash in the Keil IDE.

1.5.2.2. J-Flash Programming

Required tools: J - Link.exe version 6.2 or higher, FR8000.FLM.

1. Add the chip declaration to the JLinkDevices.xml:

```
<!-- -->
<!-- FreqChip -->
<!-- -->
<Device>
  <ChipInfo Vendor="FreqChip" Name="FR8000" Core="JLINK_CORE_CORTEX_M3"
  WorkRAMAddr="0x11000000" WorkRAMSize="0x4000"/>
  <FlashBankInfo Name="External Flash" BaseAddr="0x10000000" MaxSize="0x80000"
  Loader="Devices\Freqchip\FR8000.FLM" LoaderType="FLASH_ALGO_TYPE_OPEN" AlwaysPresent="1"/>
</Device>
```

2. Create a SEGGER\JLink\Devices\Freqchip folder in the JLink installation directory and copy the FR8000.FLM file to this folder:



Figure 1-11 Storage Location of 10FR8000.FLM

3. Open the J-Flash and create a new project

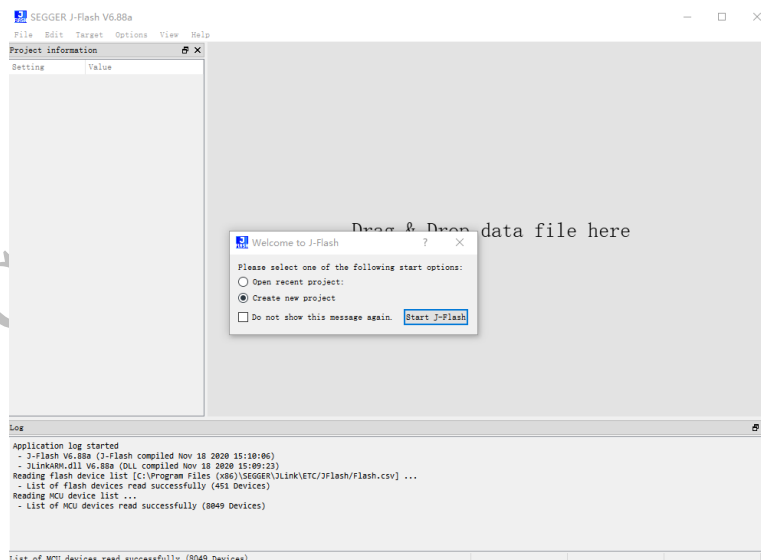


Figure 1-12 Create New Project1

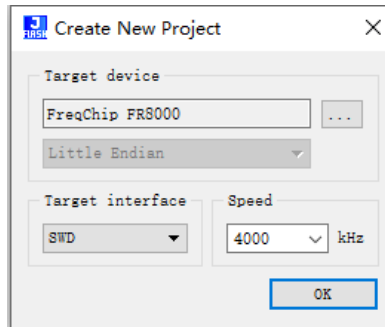


Figure 1-13 Create New Project1

4. Select FR8000 as target device

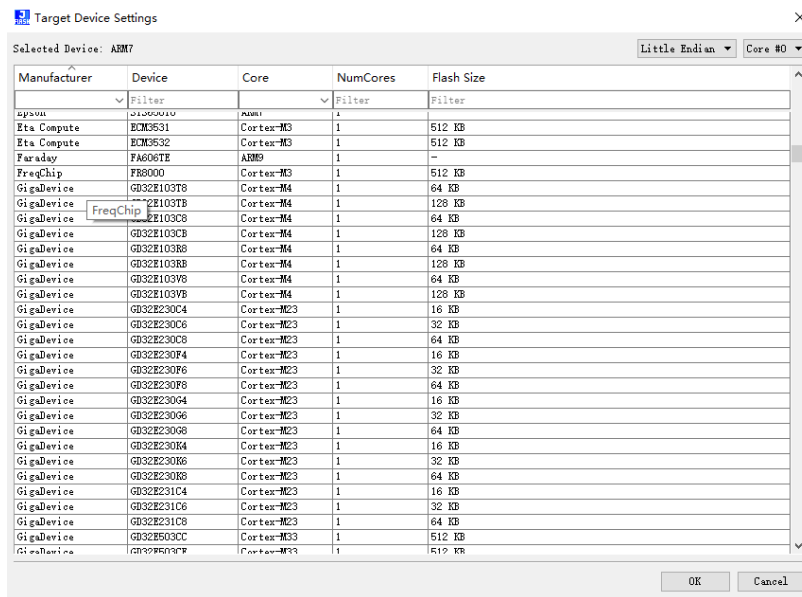


Figure 1-14 target device selection

5. Enter the following interface, and select "Target -> Connect" from the menu bar to connect the chip to be programmed

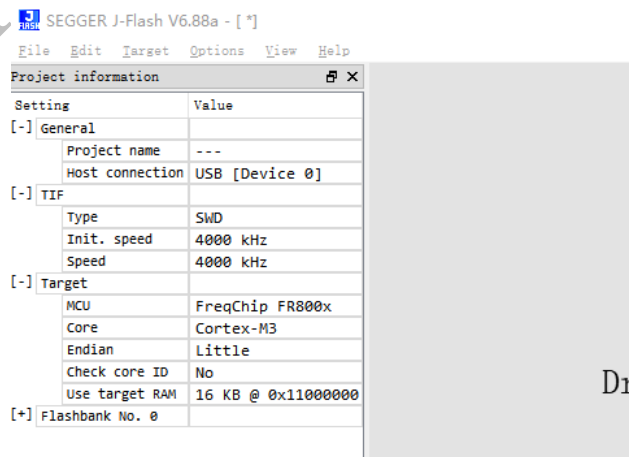


Figure 1-15 Connection

6. Select "File -> Open data file" from the menu bar, and enter the starting programming address 0x10000000, then open the file to be programmed
7. Select "Target -> Production Programming" or "Target -> Manual Programming -> Program/Program & Verify" from the menu bar to perform the programming of the file.

Freqchip Confidential

2. BLE Protocol Stack

The FR8000 SDK includes a complete BLE protocol stack. The controller is implemented in ROM and the host part is implemented in lib. It provides a set of rich APIs for users to call to complete Bluetooth functions.

2.1. Initialization

2.1.1. Protocol Stack Initialization

Before performing Bluetooth operations, you need to call the `ble_stack_init` to initialize the protocol stack. Since initialization involves Bluetooth address and buffer allocation of Bluetooth link layer, you need to call the following two functions to set this content before calling `ble_stack_init`:

1. Set the Bluetooth address:

```
mac_addr_t mac_addr;
mac_addr.addr[0] = 0xbd;
mac_addr.addr[1] = 0xad;
mac_addr.addr[2] = 0x10;
mac_addr.addr[3] = 0x11;
mac_addr.addr[4] = 0x20;
mac_addr.addr[5] = 0x20;
gap_address_set(&mac_addr, BLE_ADDR_TYPE_PRIVATE);
```

2. Configure the buffer size and number of the link - layer transmit and receive

```
ble_stack_configure(true,
    BLE_STACK_ENABLE_CONNECTIONS,
    BLE_STACK_RX_BUFFER_CNT,
    BLE_STACK_RX_BUFFER_SIZE,
    BLE_STACK_TX_BUFFER_CNT,
    BLE_STACK_TX_BUFFER_SIZE,
    BLE_STACK_ADV_BUFFER_SIZE);
```

The transmission and reception of Bluetooth data are realized through a shared RAM storage space between software and hardware. This space is 8 KB in size. The actual space used is related to factors such as the number of supported links, the size and quantity of transmitted and received packets. Therefore, when configuring these parameters, it is necessary to ensure that the actual RAM size is not exceeded.

2.1.2. GAP callback function

Messages such as successful device connection, parameter update, and discovery of broadcast packets belong to GAP messages. The application layer needs to register a receiving function to handle these messages: `gap_set_cb_func (proj_ble_gap_evt_func)`. An example of this callback function is as follows:

```
void proj_ble_gap_evt_func(gap_event_t *event)
{
    switch(event->type) {
        case GAP_EVT_ADV_END:
            ...
            break;
        case GAP_EVT_SCAN_END:
            ...
            break;
        case GAP_EVT_ADV_REPORT:
            if(memcmp(event->param.adv_rpt-
>src_addr.addr.addr, "\xfd\x37\xe3\xe1\xfc\x02", 6)==0) {
                ...
            }
            break;
        case GAP_EVT_ALL_SVC_ADDED:
            ...
            break;
        case GAP_EVT_MASTER_CONNECT:
            master_link_conidx = (event->param.master_connect.conidx);
            break;
        case GAP_EVT_SLAVE_CONNECT:
            slave_link_conidx = event->param.slave_connect.conidx;
            break;
        ...
    }
}
```

2.2. Advertising

2.2.1. Legacy Advertising

When the device initiates the legacy advertising process, the interaction process between the application layer and the BLE protocol stack is as follows:

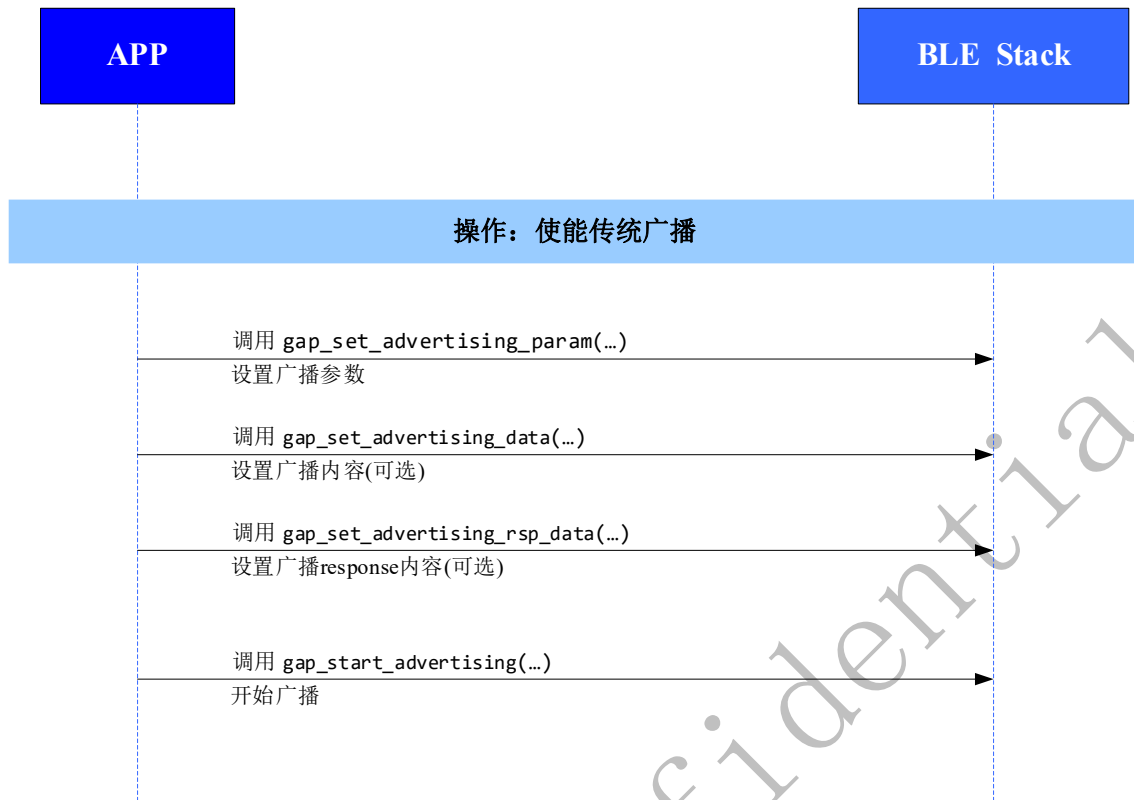


Figure 2-1 Enables Legacy Advertising Process

2.2.1.1. Set Advertising Parameters

The following is an example of how to set broadcast parameters by configuring a normal advertising that can be discovered, linked, and has an advertising interval of 40ms:

```

gap_adv_param_t adv_param;
adv_param.adv_mode = GAP_ADV_MODE_UNDIRECT;
adv_param.disc_mode = GAP_ADV_DISC_MODE_GEN_DISC;
adv_param.adv_addr_type = GAP_ADDR_TYPE_STATIC;
adv_param.adv_chnl_map = GAP_ADV_CHAN_ALL;
adv_param.adv_filt_policy = GAP_ADV_ALLOW_SCAN_ANY_CON_ANY;
adv_param.adv_intv_min = 0x40;
adv_param.adv_intv_max = 0x40;
gap_set_advertising_param(&adv_param);
    
```

For a advertising that has already started, the user needs to pause the current broadcast before reconfiguring the parameters, otherwise the configuration will fail.

2.2.1.2. Set Advertising Content

When the `adv_mode` field in the advertising parameter setting is configured as `GAP_ADV_MODE_DIRECT` or `GAP_ADV_MODE_HDC_DIRECT`, there is no need to set the advertising data. In other cases, the broadcast content needs to be set. The maximum data segment of legacy advertising is 31 bytes, in which the flag field is generated by the host protocol stack according to advertising parameters and occupies 3 bytes, so the data length left for user configuration is 28 bytes. The advertising content needs to be filled in strictly according to the length, type, and data formats specified in the Bluetooth protocol. Otherwise, the setting will fail.

```
uint8_t adv_data[0x1C];
uint8_t *pos;
uint8_t adv_data_len = 0;
pos = &adv_data[0];
uint8_t manufacturer_value[] = {0x00, 0x00};
*pos++ = sizeof(manufacturer_value) + 1;
*pos++ = '\xff';
memcpy(pos, manufacturer_value, sizeof(manufacturer_value));
pos += sizeof(manufacturer_value);

uint16_t uuid_value = 0x1812;
*pos++ = sizeof(uuid_value) + 1;
*pos++ = '\x03';
memcpy(pos, (uint8_t *)&uuid_value, sizeof(uuid_value));
pos += sizeof(uuid_value);
adv_data_len = ((uint32_t)pos - (uint32_t)(&adv_data[0]));

gap_set_advertising_data(adv_data, adv_data_len );
```

When the advertising content needs to be updated, the user does not need to pause the current broadcast, and can directly call this function to update the content.

2.2.1.3. Set Scan Response Content

The scan response content is the response made by the device when it receives a scan request during the advertising process. When the `adv_mode` field in the advertising parameter settings is `GAP_ADV_MODE_UNDIRECT` or `GAP_ADV_MODE_NON_CONN_SCAN`, the scan response data needs to be set. In other cases, it does not need to be set. In traditional broadcasting, the scan response content can

be configured with a maximum length of 31 bytes. The content of the scan response needs to be filled in strictly according to the length, type, and data formats specified in the Bluetooth protocol. Otherwise, the setting will fail.

```
uint8_t scan_rsp_data[0x1F];
uint8_t scan_rsp_data_len = 0;
uint8_t local_name[] = "8000_ADV";
uint8_t local_name_len = sizeof(local_name);
pos = &scan_rsp_data[0];
*pos++ = local_name_len + 1; //pos len; (payload + type)
*pos++ = '\x09'; //pos: type
memcpy(pos, local_name, local_name_len);
pos += local_name_len;
scan_rsp_data_len = ((uint32_t)pos - (uint32_t)(&scan_rsp_data[0]));
gap_set_advertising_rsp_data(scan_rsp_data, scan_rsp_data_len);
```

When the scan response content needs to be updated, the user does not need to pause the current advertising and can directly call this function to update the content.

2.2.1.4. Start Advertising

The gap_start_advertising function is called to start advertising. The input parameter is the duration of advertising, with the unit of 10ms. When it is set to 0, it means that the advertising will not stop automatically.

2.2.2. Extended Advertising

The setting process of extended advertising is the same as that of legacy advertising. The main difference between the two is the setting of legacy advertising parameters.

2.2.2.1. Set Advertising Parameters

The following is an example of setting up a non-directional, connectable, 40ms interval extended advertising on the CODED PHY to demonstrate how to configure the parameters of the extended advertising:

```
gap_adv_param_t adv_param;
adv_param.adv_mode = GAP_ADV_MODE_EXTEND_CONN_UNDIRECT;
adv_param.disc_mode = GAP_ADV_DISC_MODE_NON_DISC;
adv_param.adv_addr_type = GAP_ADDR_TYPE_STATIC;
```

```
adv_param.adv_chnl_map = GAP_ADV_CHAN_ALL;
adv_param.adv_filt_policy = GAP_ADV_ALLOW_SCAN_ANY_CON_ANY;
adv_param.adv_intv_min = 0x40;
adv_param.adv_intv_max = 0x40;
adv_param.phy_mode = GAP_PHY_CODED; //GAP_PHY_2MBPS
adv_param.adv_sid = 0x2;
gap_set_advertising_param(&adv_param);
```

Among them:

1. The adv_mode parameter is used to set the type of advertising. For specific details, please refer to the macro definitions of the broadcasting types in gap_api.h;
2. The phy_mode is used to set the communication rate adopted for advertising. If it is configured as GAP_PHY_CODED, both the primary and secondary will adopt the CODED PHY. If it is configured as GAP_PHY_1MBPS, both the primary and secondary will adopt the 1M PHY. If it is configured as GAP_PHY_2MBPS, the primary will adopt the 1M PHY and the secondary will adopt the 2M PHY;
3. The adv_sid parameter represents the set_id number specified by the application layer of this advertising set, and the range is from 0 to 0xF.

2.2.2.2. Set Advertising Content

When the adv_mode is set to GAP_ADV_MODE_EXTEND_NON_CONN_SCAN or GAP_ADV_MODE_EXTEND_NON_CONN_SCAN_DIRECT, advertising data needs to be set. The remaining types do not need to be set:

```
uint8_t adv_data[0x1C];
uint8_t local_name[] = "8000_EXT_ADV";
uint8_t local_name_len = sizeof(local_name);
uint8_t *pos;
uint8_t adv_data_len = 0;
pos = &adv_data[0];
*pos++ = local_name_len + 1; //pos len; (payload + type)
*pos++ = '\x09'; //pos: type
memcpy(pos, local_name, local_name_len);
pos += local_name_len;
adv_data_len = ((uint32_t)pos - (uint32_t)&adv_data[0]);
gap_set_advertising_data(adv_data, adv_data_len);
```

The maximum length of the extended advertising supported by the protocol stack is dependent on the configuration of BLE_STACK_ADV_BUFFER_SIZE in the protocol stack

initialization. When the value is set to be less than 251, the maximum length is consistent with the value, and if it is equal to 251, a length of 251*4 bytes can be supported.

When the advertising content needs to be updated, the current advertising needs to be stopped first to avoid insufficient underlying buffer and resulting in update failure.

2.2.2.3. Set Scan Response Content

When the `adv_mode` is set to `GAP_ADV_MODE_EXTEND_NON_CONN_SCAN` or `GAP_ADV_MODE_EXTEND_NON_CONN_SCAN_DIRECT`, advertising data needs to be set. The remaining types do not need to be set:

```
uint8_t scan_rsp_data[0x1F];
uint8_t scan_rsp_data_len = 0;

uint8_t local_name[] = "8000_SCAN_RSP";
uint8_t local_name_len = sizeof(local_name);
pos = &scan_rsp_data[0];
*pos++ = local_name_len + 1; //pos len; (payload + type)
*pos++ = '\x09'; //pos: type
memcpy(pos, local_name, local_name_len);
pos += local_name_len;
scan_rsp_data_len = ((uint32_t)pos - (uint32_t)(&scan_rsp_data[0]));
gap_set_advertising_rsp_data(scan_rsp_data, scan_rsp_data_len);
```

The maximum length of the extended advertising supported by the protocol stack is dependent on the configuration `BLE_STACK_ADV_BUFFER_SIZE` in the protocol stack initialization. When the value is set to be less than 251, the maximum length is the same as this value. If it is equal to 251, a length of up to 251 * 4 bytes can be supported.

When the broadcasting content needs to be updated, the current broadcasting needs to be stopped first to avoid insufficient underlying buffer, which may cause the update to fail.

2.2.2.4. Start Advertising

The method of starting extended advertising is the same as that of starting legacy advertising. Please refer to 2.2.1.4.

2.2.3. Periodic Advertising

When the device starts the periodic advertising process, the interaction process between the application layer and the BLE protocol stack is shown in the following figure:

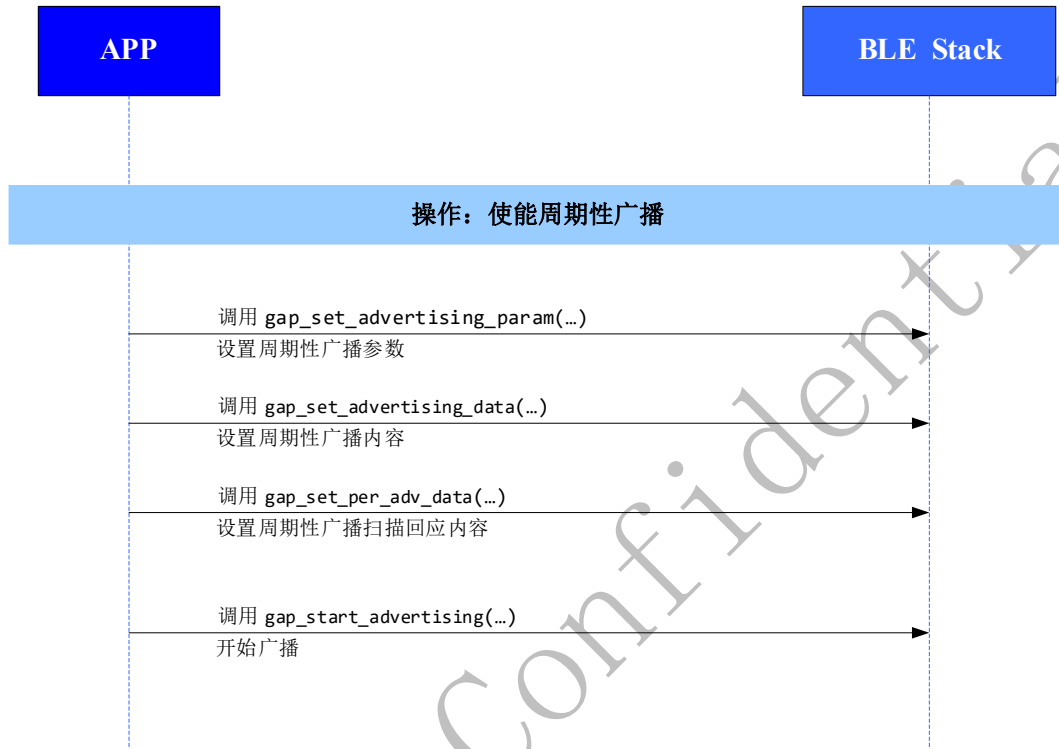


Figure 2-2 Start Periodic Advertising Process

2.2.3.1. Set Advertising Parameters

```

gap_adv_param_t adv_param;
adv_param.adv_mode = GAP_ADV_MODE_PER_ADV_UNDIRECT;
adv_param.disc_mode = GAP_ADV_DISC_MODE_GEN_DISC;
adv_param.adv_addr_type = GAP_ADDR_TYPE_STATIC;
adv_param.adv_chnl_map = GAP_ADV_CHAN_ALL;
adv_param.adv_filt_policy = GAP_ADV_ALLOW_SCAN_ANY_CON_ANY;
adv_param.adv_intv_min = 0x40;
adv_param.adv_intv_max = 0x40;
adv_param.phy_mode = GAP_PHY_1MBPS;
adv_param.adv_sid = 9;
adv_param.per_adv_intv_min = 0x100;
adv_param.per_adv_intv_max = 0x100;
gap_set_advertising_param(&adv_param);
    
```

Among them:

1. `per_adv_intv_min` and `per_adv_intv_max` indicate the minimum and maximum time intervals of the periodic advertising content after the periodic advertising is established, and the unit is 1.25ms.
2. Other parameters have the same meanings as described in section 2.2.2.1

2.2.3.2. Set Advertising Content

The method for setting the advertising content is the same as that described in 2.2.2.2.

2.2.3.3. Set Periodic Advertising Data

Data for periodic advertising is set in the following way:

```
uint8_t per_adv_data1[] = "\x09\x09\x11\x12\x13\x14\x15\x16\x17\x18";  
gap_set_per_adv_data(per_adv_data1, sizeof(per_adv_data1));
```

2.2.3.4. Start Advertising

The method of starting extended advertising is the same as that of starting legacy advertising. Please refer to 2.2.1.4.

2.2.4. Related GAP Events

There are mainly the following two types of GAP events related to advertising:

There are mainly the following two types of GAP events related to broadcasting:

1. `GAP_EVT_ADV_END`: This event will occur in three cases: when `gap_stop_advertising` is actively called, when the broadcasting times out, or when the device is connected.
2. `GAP_EVT_SLAVE_CONNECT`: After a device that is broadcasting connectable data is connected, the GAP callback function of the application layer will receive this event.

2.3. Scan

2.3.1. Traditional scanning

When the device starts the scanning process, the interaction process between the application layer and the BLE protocol stack is as follows:



Figure 2-3 Legacy Scanning

The following sample code demonstrates how to start normal active scanning:

```

gap_scan_param_t scan_param;
scan_param.scan_mode = GAP_SCAN_MODE_GEN_DISC;
scan_param.dup_filt_pol = 0;
scan_param.scan_intv = 32; //scan event on-going time
scan_param.scan_window = 20;
scan_param.duration = 1500; //15s
scan_param.phy_mode = GAP_PHY_1MBPS;
gap_start_scan(&scan_param);
    
```

Among them:

1. scan_mode indicates the scanning mode: GAP_SCAN_MODE_GEN_DISC represents active scanning. In this mode, the scan request data packet will be sent according to the ADV type to obtain the scan response; GAP_SCAN_MODE_OBSERVER represents passive scanning. In this mode, it will only be in a passive listening state.
2. dup_filt_pol indicates whether to perform duplicate filtering check before reporting the scanned content.

3. `scan_window` and `scan_intv` represent the scanning window and scanning interval respectively, with the unit of 625us.
4. `Duration` represents the duration of the entire scanning action. It will automatically stop after the time is up, with the unit of 10ms. If it is set to 0, it means that the scanning action will not stop automatically.
5. For legacy scanning, `phy_mode` is set to `GAP_PHY_1MBPS`.

2.3.2. Extended Scan

The startup process and calling method of the extended advertising are the same as those of traditional scanning described in 2.3.1. When scanning the primary extended advertising using `CODEC_PHY`, the `phy_mode` parameter needs to be set to `GAP_PHY_CODED`.

2.3.3. Synchronous Scanning



Figure 2-4 Periodic Advertising Synchronization Scan Flow

2.3.3.1. Set Synchronization Establishment Parameters

The following code can be used to set synchronization parameters:

```

gap_per_sync_param_t per_sync_param;
per_sync_param.sup_to = 600;
per_sync_param.adv_sid = 9;
per_sync_param.adv_dev_addr.addr_type = 0;
memcpy(per_sync_param.adv_dev_addr.addr, "\x1F\x19\x07\x09\x17\x20", MAC_ADDR_LEN);
gap_start_per_sync(&per_sync_param);
    
```

Among them:

1. `sup_to` indicates that after the periodic advertising synchronization is established, if no advertising data is received for a continuous period of time, a handshake timeout will occur to stop the synchronization action, and the unit is 10ms.
2. The `adv_sid` parameter indicates the `set_id` number of the periodic advertising to which the synchronization action is directed.
3. `adv_dev_addr` indicates the address and address type of the synchronization object.

2.3.3.2. Start Scanning

The startup method of synchronous scanning is the same as that described in Section 2.3.2. When a primary channel advertising packet that meets the parameter requirements is scanned, the periodic advertising synchronization establishment event `GAP_EVT_PER_SYNC_ESTABLISHED` will be generated. Subsequent extended advertising packets and periodic advertising data will be uploaded through `GAP_EVT_ADV_REPORT`

2.3.4. GAP Events

The GAP events related to scanning mainly include the following:

1. `GAP_EVT_ADV_REPORT`, which is used to report the scanned advertising data
2. `GAP_EVT_SCAN_END`, which will be generated when the function to stop scanning, `gap_stop_advertising` is called or when the scanning time expires.
3. `_ESTABLISHED GAP_EVT_PER_SYNC`, which indicates that the synchronous scan was established successfully.
4. `GAP_EVT_PER_SYNC_END`, which indicates that the synchronization has ended due to a timeout.

2.4. Connection

2.4.1. Connect To Legacy Advertising

When a device initiates a legacy advertising connection process, an interaction process between an application layer and a BLE protocol stack is shown in the following figure. This active connection action can only connect to connectable traditional broadcasts. The broadcast types are: `GAP_ADV_MODE_UNDIRECT`, `GAP_ADV_MODE_DIRECT`, and `GAP_ADV_MODE_HDC_DIRECT`.

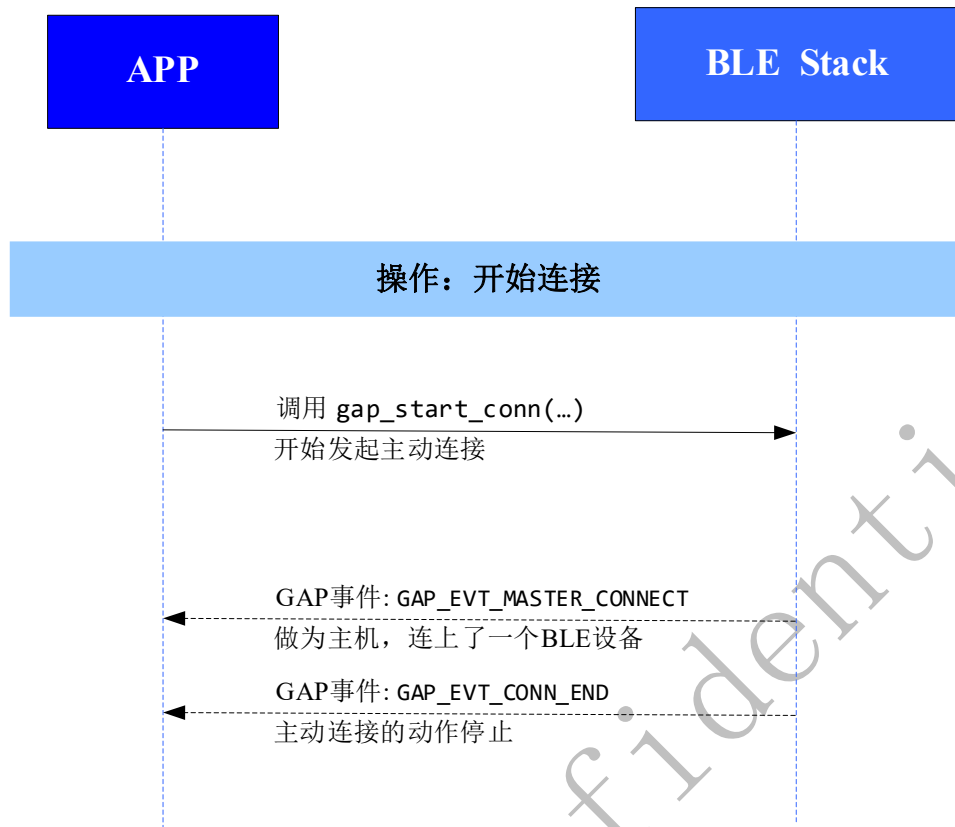


Figure 2-5 Connection Establishment Process

The following code shows how to initiate a legacy connection:

```
void gap_start_conn(mac_addr_t *addr, uint8_t addr_type, uint16_t min_itvl, uint16_t max_itvl,
uint16_t slv_latency, uint16_t timeout);
addr_t mac = {{0x1F, 0x19, 0x07, 0x09, 0x17, 0x20}};
gap_start_conn(&mac, 0, 9, 9, 0, 400);
```

Among them:

1. Addr represents the MAC address of the BLE device to be connected.
2. addr_type represents the MAC address type of the BLE device to be connected. 0 represents the public address; 1 represents the private address.
3. min_itvl and max_itvl represent the basic handshake interval time adopted after the connection is established. Unit: 1.25ms.
4. slv_latency represents the latency parameter adopted by the slave device after the connection is established.
5. Timeout represents the timeout disconnection time adopted by the slave device after the connection is established. Unit: 10ms.

2.4.2. Connection Extended Broadcast

When the device initiates an extended advertising connection process, an interaction process between the application layer and the BLE protocol stack is the same as that of connecting to legacy advertising. The active connection action can only connect extension advertising of connectable types. The advertising types are:

GAP_ADV_MODE_EXTEND_CONN_UNDIRECT and
GAP_ADV_MODE_EXTEND_CONN_DIRECT.

1. The primary channel uses 1 MBPS or 2 MBPS

When the primary channel of the extended advertising uses 1 MBPS or 2 MBPS, the interaction process is exactly the same as that in 2.4.1.

2. Primary channel with CODED PHY

When the primary channel of the extended advertising uses CODED PHY, the system provides a function to establish a connection. The meaning of the parameters it uses is consistent with that of `gap_start_conn`.

```
void gap_start_conn_long_range(mac_addr_t *addr, uint8_t addr_type, uint16_t min_itvl, uint16_t max_itvl, uint16_t slv_latency, uint16_t timeout);
```

2.4.3. Other

1. Disconnect

The following function can be used to disconnect an established connection. This function is applicable to work in master or slave mode.

```
void gap_disconnect_req(uint8_t conidx);
```

2. Cancel the current connection

During the connection establishment process, the user can call the following function to cancel the connection.

```
void gap_stop_conn(void);
```

2.4.4. Related GAP Events

The GAP events related to connection mainly include the following:

1. **GAP_EVT_MASTER_CONNECT**: After initiating an active connection to a traditional broadcast or an active connection to an extended broadcast and connecting to the peer BLE device, the protocol stack will generate an event indicating that the active connection has been successfully established.
2. **GAP_EVT_CONN_END**, in the following two cases: after calling the stop active connection function or establishing the connection, the active connection action stop event will be generated.

2.5. Definition of Profile

The Generic Attribute Profile(GATT) is used for data communication between two connected devices. In the BLE GATT layer, data is transmitted and stored in the form of Characteristic.

2.5.1. GATT Roles

From the perspective of GATT, when two devices are in a connected state, one device acts as a GATT server and the other device acts as a GATT client. This role assignment has nothing to do with the device roles (peripheral device, central device) in the GAP layer. A peripheral device can act as either a GATT client or a GATT server; a central device can also act as either a GATT client or a GATT server.

- **GATT client**: The device initiates commands, requests and receives responses, notifications and indications.
- **GATT server**: The device receives commands, requests, and sends responses, notifications, and instructions.

2.5.2. GATT Profile Hierarchy

A profile can contain one or more services; a service can contain one or more characteristics; a characteristic contains at least two attributes, including the characteristic declaration and the characteristic value. The following sections will describe attributes, characteristics, and services in detail.

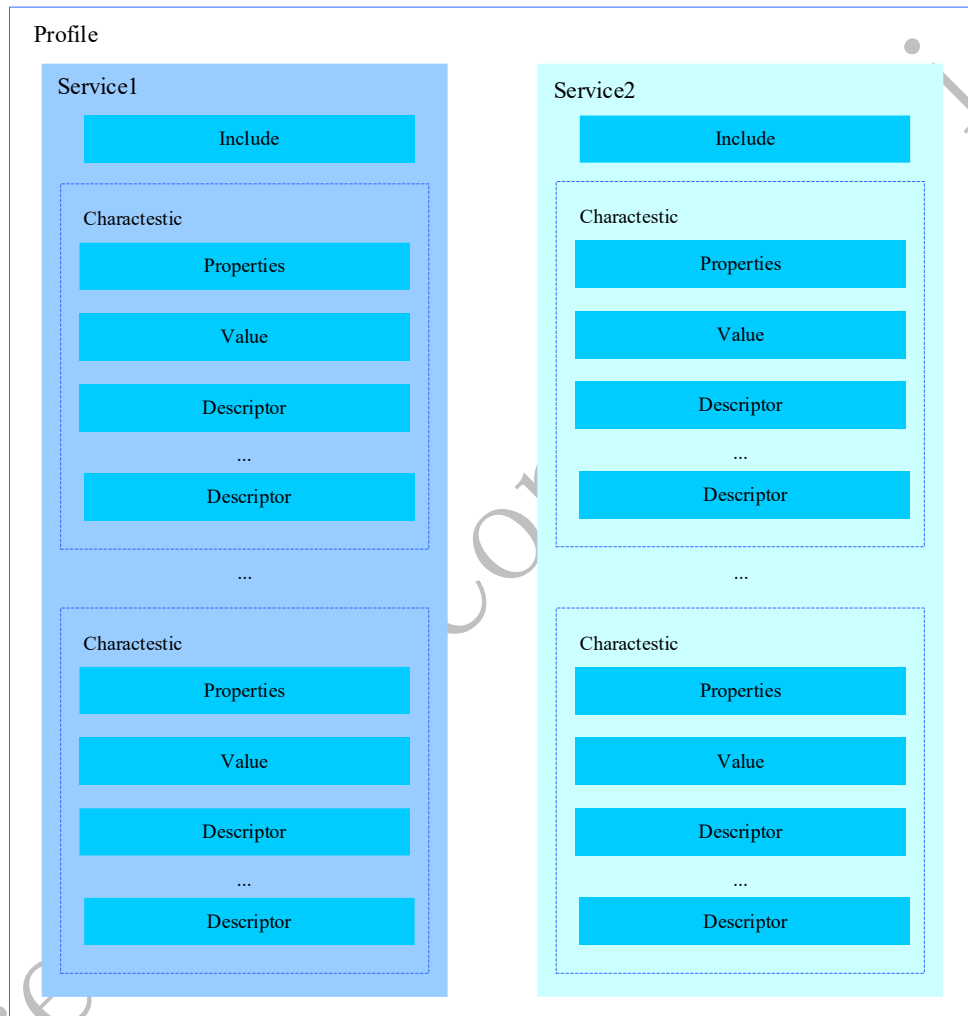


Figure 2-6 Service, Characteristic, Attribute Hierarchy Relationship

2.5.2.1. Attribute

In Bluetooth Low Energy, a characteristic can be regarded as a collection of a set of attribute information, including the characteristic declaration, the characteristic value, and the

characteristic descriptor. Data is interacted in the form of attributes. Attributes mainly include the following parts:

- **Attribute Handle:** The handle is the index of the attribute in the GATT attribute table, and each attribute has a unique handle.
- **Attribute Type:** It represents the thing that the data represents, usually a UUID (Universally Unique Identifier) specified by the Bluetooth SIG or customized by the user.
- **Attribute Value:** The data value of the attribute.
- **Attribute Permissions:** It stipulates the access permissions of the GATT client device to the attribute, including whether it can access and how to access.

2.5.2.2. Characteristic

A typical characteristic consists of the following attributes:

- **Characteristic declaration:** describes the nature, storage location (handle), and type of the characteristic.
- **Characteristic value:** the data value of the characteristic.
- **Characteristic descriptor:** Additional information or configuration describing a characteristic.

2.5.2.3. Service

A profile contains one or more services. A GATT service is a collection of a series of characteristics. For example, the heart rate service includes characteristics such as heart rate measurement and body position. Common services include:

- **Generic Access Service:** This service includes device and access information, such as device name, vendor identifier, product identifier, etc. The characteristics defined by this service include: Device Name, Appearance, Peripheral Preferred Connection Parameters, etc.
- **Generic Attribute Service:** This service is mainly used by the server to notify the connected client that the services it provides have changed. This service contains the Service changed characteristic.

The above two services are added to the server's database by default when the BLE protocol stack is initialized. However, the user can call the `gatt_delete_svc` function to delete these two default services after the protocol stack is initialized, and then reference the gap and gatt services defined in SDK folder\components\ble\profiles\ble_gap & ble_gatt. In this way, the user can freely change the definitions of these two services.

2.5.3. Attribute Table

The communication of Bluetooth Low Energy is achieved by performing operations such as reading and writing on attributes. This section introduces how to define these attributes to form a complete service.

2.5.3.1. Definition of Attribute Table

Each Service must define an attribute table and pass it to the protocol stack through the initialization function of the profile. An instance of the batt service is used to explain the definition of the attribute table. The code of this service is in SDK_Folder\components\profiles\ble_batt\batt_service.c. The defined attribute table array is as follows:

```
enum
{
    IDX_BATT_SERVICE,
    IDX_BATT_LEVEL_CHAR_DECLARATION,
    IDX_BATT_LEVEL_CHAR_VALUE,
    IDX_BATT_LEVEL_CCCD,
    IDX_BATT_NB,
};
typedef struct
{
    uint8_t size;           //!< Length of UUID
    uint8_t p_uuid[16];    //!< Pointer to uuid, could be 2 or 16 bytes array.
} gatt_uuid_t;
typedef struct
{
    gatt_uuid_t    uuid;    //!< Attribute UUID
    uint16_t       prop;    //!< Attribute properties, see @GATT_PROP_BITMAPS_DEFINES
    uint16_t       max_size; //!< Attribute data maximum size
    uint8_t        *p_data; //!< Attribute data pointer
} gatt_attribute_t;
static const uint8_t batt_svc_uuid[UUID_SIZE_2] = UUID16_ARR(BATT_SERV_UUID);
static const gatt_attribute_t batt_att_table[] =
{
    [IDX_BATT_SERVICE] = { { UUID_SIZE_2, UUID16_ARR(GATT_PRIMARY_SERVICE_UUID)},
        GATT_PROP_READ,  UUID_SIZE_2, (uint8_t *)batt_svc_uuid,
    },
    [IDX_BATT_LEVEL_CHAR_DECLARATION] = { { UUID_SIZE_2, UUID16_ARR(GATT_CHARACTER_UUID)},
        GATT_PROP_READ, 0, NULL,
    },
    [IDX_BATT_LEVEL_CHAR_VALUE] = { { UUID_SIZE_2, UUID16_ARR(BATT_LEVEL_UUID)},
        GATT_PROP_READ | GATT_PROP_NOTI, sizeof(uint8_t), NULL,
    },
    [IDX_BATT_LEVEL_CCCD] = { { UUID_SIZE_2, UUID16_ARR(GATT_CLIENT_CHAR_CFG_UUID)},
        GATT_PROP_READ | GATT_PROP_WRITE, sizeof(uint16_t), NULL,
    },
};
```

Explanation:

- The attribute table `batt_att_table[]` is an array. The variable type of the array is the structure type `gatt_attribute_t`. Each variable in the array constitutes an attribute in the attribute table.
- The explanation of each element in the structure type `gatt_attribute_t` is as follows:
 - a) `uuid` represents the UUID of this attribute. `uuid.size` is the length of the UUID, and the value can be 16 bits or 128 bits, represented by the macros `UUID_SIZE_2` and `UUID_SIZE_16`. `uuid.p_uuid` is the content of the UUID.
 - b) `prop` represents the read and write permissions of this attribute. For the macro definitions of profile attribute permissions, please refer to the definitions in the macro definition group `GATT_PROP_BITMAPS_DEFINES` in `gatt_api.h`, as follows:

```
#define GATT_PROP_BROADCAST      (1<<0)  //!< Attribute is able to broadcast
#define GATT_PROP_READ          (1<<1)  //!< Attribute is Readable
#define GATT_PROP_WRITE_CMD     (1<<2)  //!< Attribute supports write with no response
#define GATT_PROP_WRITE_REQ     (1<<3)  //!< Attribute supports write request
#define GATT_PROP_NOTI          (1<<4)  //!< Attribute is able to send notification
#define GATT_PROP_INDIC         (1<<5)  //!< Attribute is able to send indication
#define GATT_PROP_AUTH_SIG_WRTIE (1<<6)  //!< Attribute supports authenticated signed write
#define GATT_PROP_EXTEND_PROP   (1<<7)  //!< Attribute supports extended properities
#define GATT_PROP_WRITE         (1<<8)  //!< Attribute is Writable (write_req and write_cmd)
#define GATT_PROP_AUTHEN_READ   (1<<9)  //!< Read requires Authentication
#define GATT_PROP_AUTHEN_WRITE  (1<<10) //!< Write requires Authentication
```

The commonly used read and write permissions are from line 2 to line 6, and from line 9 to line 11. Multiple permissions can be set for an attribute by using the OR operator. For example: `GATT_PROP_READ | GATT_PROP_NOTI` means that the attribute supports both ntf and read.

- c) `max_size` represents the maximum data length supported by this attribute.
 - d) `p_data` represents a pointer to the data to be processed by this attribute.
- When assigning values to the array variables of the attribute table, there are the following combinations of `max_size` and `p_data`, and their meanings are as follows.
 - a) `max_size > 0`, `p_data != NULL`. Memory will be allocated internally when the profile is created, and the data pointed to by `p_data` will be copied to the internal memory.

- b) `max_size > 0, p_data == NULL`. No memory will be allocated internally when the profile is created. If the peer reads the value of this attribute, a read operation event callback will be generated.
- c) `max_size == 0, p_data == NULL`. This attribute cannot be read, or reading this attribute will return Null.
- d) `max_size == 0, p_data != NULL`. This situation cannot occur.
- When assigning values to the attribute variable with the UUID of `GATT_PRIMARY_SERVICE_UUID`, `max_size` and `p_data` represent the length of the service group UUID and the content of the UUID respectively. In the example attribute table, the `max_size` of this attribute element is `UUID_SIZE_2`, and `p_data` is `(uint8_t *)batt_svc_uuid`. This means that the UUID length of this service group is 16 bits, and the UUID value is the value of `batt_svc_uuid`.

2.5.3.2. Create Attribute Table

When the device is powered on or restarted, the application layer needs to build its own service list during the initialization phase. Each service contains some attributes, and the callback function of the service is defined according to its own functional requirements. The attribute table and the callback function are passed to the protocol stack when the service addition function is called. For example, in the application layer initialization function, the `svcA_gatt_add_service` and `svcB_gatt_add_service` functions are called in sequence to add two custom services. The execution process is shown in the following figure.

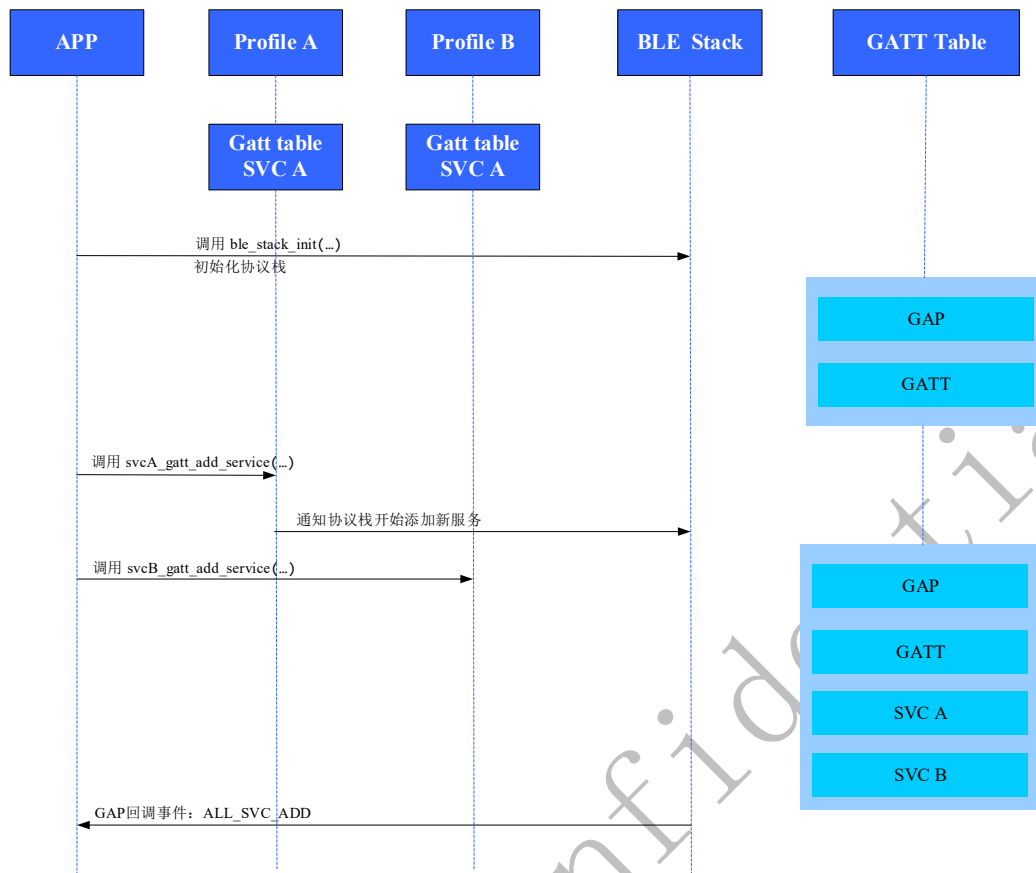


Figure 2-7 The process of establishing the attribute table

2.5.4. Basic Process of GATT Client

2.5.4.1. Client

The GATT client mainly initiates read and write commands and requests to the server and receives responses, instructions, and notifications from the server.

The GATT client does not have an attribute table. It is mainly used to obtain attribute information rather than provide attribute information and services. The read and write operations on the attributes of the attribute table are completed by using the API interfaces defined in gatt_api.h. After most of the GATT client APIs are called, the results can be returned through the client event callback function, including the read attribute value, the completion status of writing, indications, etc. For a detailed description of the APIs, please refer to the function definition description in gatt_api.h. The basic process of the client GATT is shown in the following figure.

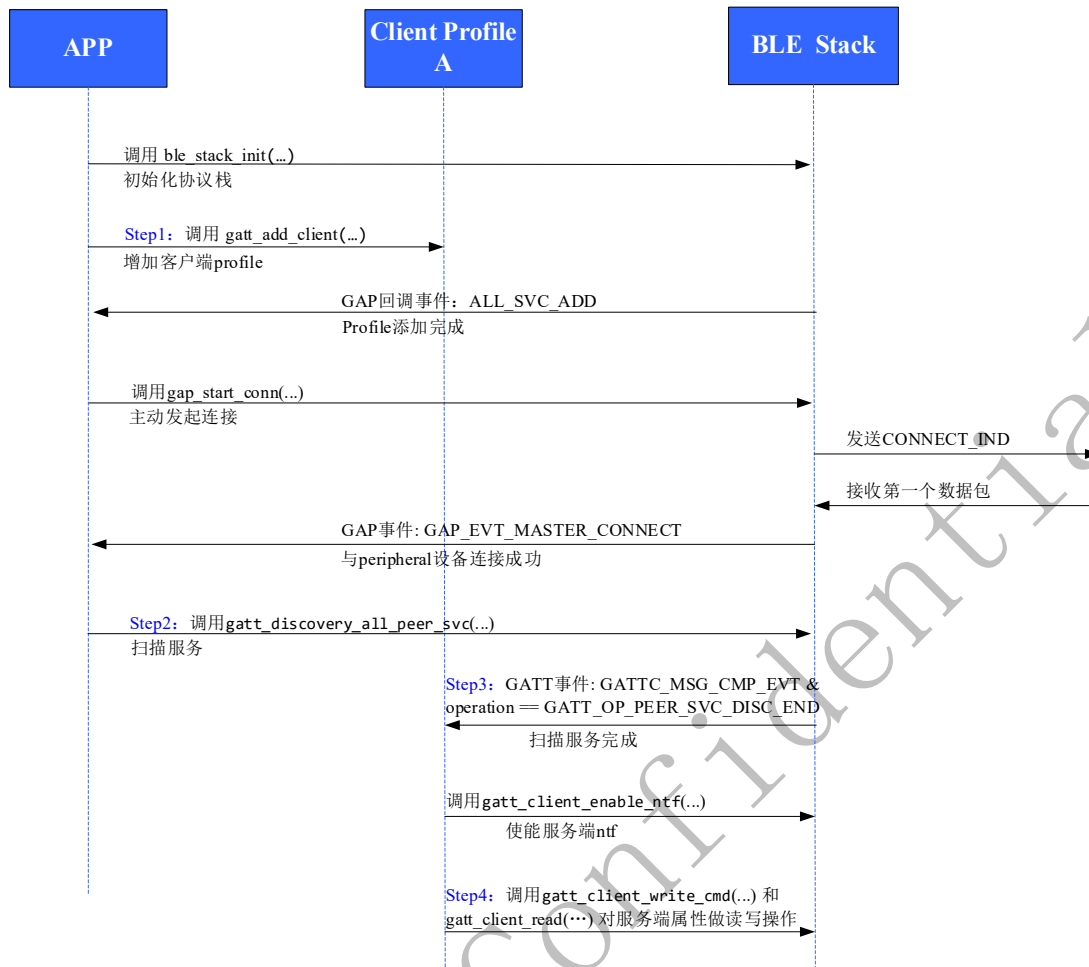


Figure 2-8 Client Workflow in SDK

2.5.4.2. Increase client profile

Although the client does not have an attribute table, in the protocol stack implementation of FR8000, in order to simplify the service discovery process of the application layer, the user needs to register an array of server UUIDs that the application layer is concerned about and a callback function for receiving messages with the protocol stack. This UUID array contains the UUIDs of the attributes that the client may initiate functions such as reading, writing, and receiving notifications. After the connection is established, the user can call the `gatt_discovery_all_peer_svc` function to control the protocol stack to discover whether the server database contains the UUIDs registered during initialization. The following is an example of client profile registration to explain

this process. The code of this client profile is in SDK_Folder\examples\none_evm\ble_simple_central\code\ ble_simple_central.c.

```
#define SP_CHAR1_UUID          0xFFFF1
#define SP_CHAR2_UUID          0xFFFF2
#define SP_CHAR3_UUID          0xFFFF4
const gatt_uuid_t client_att_tb[] =
{
    [0] = { UUID_SIZE_2, UUID16_ARR(SP_CHAR1_UUID)},
    [1] = { UUID_SIZE_2, UUID16_ARR(SP_CHAR2_UUID)},
    [2] = { UUID_SIZE_2, UUID16_ARR(SP_CHAR3_UUID)},
};
uint8_t client_id;
static uint16_t simple_central_msg_handler(gatt_msg_t *p_msg)
{
    co_printf("CCC:%x\r\n", p_msg->msg_evt);
    switch(p_msg->msg_evt)
    {
        case GATTC_MSG_NTF_REQ:
        {
            co_printf("GATTC_MSG_NTF_REQ\r\n");
        }
        break;
    }
}
void user_main(void)
{
    ...
    gatt_client_t client;
    client.p_att_tb = client_att_tb;
    client.att_nb = 3;
    client.gatt_msg_handler = simple_central_msg_handler;
    client_id = gatt_add_client(&client);
}
```

Notes:

- client_att_tb[] is an array of uuids that records the uuids of the attributes that the user really needs to use. After calling the scan service function, the protocol stack can upload the handle numbers corresponding to these uuid attributes.
- Line 11, client_id is the defined ID number for recording this client profile, and this value will be needed when performing read and write operations on the server later.
- Line 12, simple_central_msg_handler, is the event callback function corresponding to the client profile, which is called when the protocol stack needs to report the message corresponding to the profile.

- Lines 13 to 23 are the codes for the callback function to handle the messages uploaded by the protocol stack.
- Lines 27 to 31 define the variables of the client profile. After assigning the uuid array and the callback function to this variable, call the `gatt_add_client` function to add this client profile to the inside of the protocol stack.
- After the client profile is added, the protocol stack uploads the GAP event: `GAP_EVT_ALL_SVC_ADDED`.
- The addition of Profile can only be performed during initialization. Once the GAP event `GAP_EVT_ALL_SVC_ADDED` is uploaded, the user cannot add new profile by calling the `gatt_add_client` function.

2.5.4.3. Scanning Service

After the link is established, the device with the client profile added needs to scan the server to obtain the handle number corresponding to the required uuid. If the handle number corresponding to the uuid registered by the client profile can be scanned, it means that the peer service contains the attribute corresponding to this uuid. Various GATT read and write operations can be performed on this attribute. See the following sample code:

```
static void app_gap_evt_cb(gap_event_t *p_event)
{
    switch(p_event->type)
    {
        case GAP_EVT_MASTER_CONNECT:
            extern uint8_t client_id;
            gatt_discovery_all_peer_svc(client_id, p_event->param.master_encrypt_conidx);
            break;
    }
}
```

Notes:

- After the link is established, the event handling entry for the established host link is obtained by judging that the GAP event type is `GAP_EVT_MASTER_CONNECT`.
- Under this event, the function `gatt_discovery_all_peer_svc` is called to scan all services of the peer device. When the scanning is completed, the BLE protocol stack will upload the scanning results and the status of the end of scanning to the GATT

event callback function registered by the client profile. The uploaded event is a GATT event: GATT_MSG_CMP_EVT, and the operation = GATT_OP_PEER_SVC_REGISTERED.

- Refer to the next section for the GATT event callback processing corresponding to the client profile.

2.5.4.4. Client profile callback function

In order to handle the read and write operations of profile attributes, the profile needs to define its own read and write callback functions. The callback functions receive all the messages about this profile uploaded by the protocol stack. An example of a callback function of a client profile is used for explanation. The code of this callback function is in SDK_Folder\examples\none_evm\ble_simple_central\code\ble_simple_central.c.

```
static uint16_t simple_central_msg_handler(gatt_msg_t *p_msg)
{
    switch(p_msg->msg_evt)
    {
        case GATTC_MSG_NTF_REQ:
            show_reg(p_msg->param.msg.p_msg_data, p_msg->param.msg.msg_len, 1);
            break;
        case GATTC_MSG_READ_REQ:
            if(p_msg->att_idx == 2)
            {
                memcpy(p_msg->param.msg.p_msg_data, "\x7\x8\x9", 3);
                return 3;
            }
            break;
        case GATTC_MSG_READ_IND:
            show_reg(p_msg->param.msg.p_msg_data, p_msg->param.msg.msg_len, 1);
            break;
        case GATTC_MSG_CMP_EVT:
            {
                if(p_msg->param.op.operation == GATT_OP_PEER_SVC_REGISTERED)
                {
                    uint16_t att_handles[3];
                    memcpy(att_handles, p_msg->param.op.arg, 6);
                    gatt_client_enable_ntf_t ntf_enable;
                    ntf_enable.conidx = p_msg->conn_idx;
                    ntf_enable.client_id = client_id;
                    ntf_enable.att_idx = 2; //NF
                    gatt_client_enable_ntf(ntf_enable);
                }
            }
            break;
    }
    return 0;
}
```

```

}
return 0;
}

```

Notes:

- The GATT event callback function of the Profile will receive a message variable. This variable contains the type of the message, the link number, the number of the uuid array corresponding to the message, the handle number corresponding to the message, and the content of the message.
- The GATT event callback function processes the messages according to their types respectively, and the function ends with return 0.
- If the message is GATTC_MSG_NTF_REQ, it means that a notification message sent by the server device has been received. Printing processing is done in the example.
- If the message is GATTC_MSG_READ_REQ, it means that a read message sent by the server device has been received. A response is made in the example. Note: After the response is completed, the length of the response data needs to be returned, rather than return 0.
- If the message is GATTC_MSG_READ_IND, it means that after performing a read operation on the server device, the read response data sent by the server device has been received. Printing processing is done in the example.
- If the message is GATTC_MSG_CMP_EVT, it means that an operation for the client profile has been completed. p_msg->param.op.operation represents the type of the operation. There are the following types in total. Refer to the macro definition group GATT_OPERATION_NAME in gatt_api.h.

```

#define GATT_OP_NOTIFY          0x01    //!< GATT notification operation
#define GATT_OP_INDICA          0x02    //!< GATT indication operation
#define GATT_OP_PEER_SVC_REGISTERED 0x03    //!< Used with GATTC_CMP_EVT, GATT peer device
service registered
#define GATT_OP_WRITE_REQ       0x05    //!< GATT write request operation
#define GATT_OP_WRITE_CMD       0x06    //!< GATT write command operation, write
without response
#define GATT_OP_READ            0x07    //!< GATT read operation
#define GATT_OP_PEER_SVC_DISC_END 0x08    //!< Used with GATTC_CMP_EVT, GATT peer device
service discovery is ended

```

- For the client profile, the available operations are 0x03, 0x05, 0x06, 0x07, and 0x08

- a) 0x03 represents that the action of scanning the peer services in the service scanning step is completed, and the handle numbers of the scanned services are registered inside the protocol stack. At this time, the handle number pointed to by op.arg is the handle number corresponding to the uuid in the client uuid array on the server side. If the handle number is 0, it means that there is no attribute of this uuid on the server side. If it is non-zero, it means that the server side has the attribute of this uuid, and read and write operations can only be performed on the attributes with non-zero handle numbers.
- b) 0x05 indicates that the write operation (requiring a response) on the server-side attributes is completed.
- c) 0x06 indicates that the write operation (not requiring a response) on the server-side attributes is completed.
- d) 0x07 indicates that the read operation on the server-side attributes is completed.
- e) 0x08 indicates that the registration of the required peer service attributes inside the protocol stack is completed.
- If p_msg->param.op.operation is equal to 0x03, it means that the scanning of the peer services is completed. As a client, it generally needs to call the gatt_client_enable_ntf function to enable the notification (ntf) for the attributes with notification permissions at this time. In the example, after the service scanning is completed, an ntf_enable packet is sent for the attribute corresponding to the second uuid in the uuid array.

The parameter type of the client profile callback function is gatt_msg_t. The explanation is as follows:

```
typedef struct
{
    gatt_msg_evt_t  msg_evt;    //!< The event that message come with
    uint8_t         conn_idx;   //!< Connection index
    uint8_t         svc_id;     //!< service id of this message
    uint16_t        att_idx;    //!< Attribute index of in the service table
    uint16_t        handle;     //!< Attribute handle number in peer service
    union
    {
        gatt_msg_hdl_t  msg;    //!< GATT message, length, data pointer
        gatt_op_cmp_t   op;     //!< GATT operation, read, write, notification, indication
    } param;
}
```

```
} gatt_msg_t;
```

Notes:

- The `msg_evt` element is the type of the message. Refer to the definition of `gatt_msg_evt_t`. There are the following types:
 - a) `GATTC_MSG_READ_REQ`: The server profile receives a read operation from the peer.
 - b) `GATTC_MSG_WRITE_REQ`: The server profile receives a write operation (`write_req` and `write_cmd`) from the peer.
 - c) `GATTC_MSG_ATT_INFO_REQ`: The server profile receives an operation from the peer to read the length of the attribute value
 - d) `GATTC_MSG_NTF_REQ`: The client receives the ntf data sent by the peer.
 - e) `GATTC_MSG_IND_REQ`: The client receives the indication data sent by the peer.
 - f) `GATTC_MSG_READ_IND`: The client receives the data in response to the read operation sent by the peer.
 - g) `GATTC_MSG_CMP_EVT`: The profile receives a message indicating the completion of an event.
 - h) `GATTC_MSG_LINK_CREATE`: The profile is notified that a link has been established.
 - i) `GATTC_MSG_LINK_LOST`: The profile is notified that a link has been disconnected.
 - j) `GATTC_MSG_SVC_REPORT`: The client receives the scanned service information.
- The `conn_idx` element represents the link number of which link this message comes from.
- `svc_id`: indicates the id number of the profile corresponding to this message, which is returned by the profile add function.
- `att_idx` indicates the group number of the uuid array to the client for which attribute this message is directed. Indicates the group number of the att attribute table to the server.

- handle, indicates the handle number on the server side corresponding to the attribute of this message.
- param.msg, which indicates the data buffer pointer and length when data-related messages (except GATTC_MSG_CMP_EVT, GATTC_MSG_LINK_CREATE, and GATTC_MSG_LINK_LOST) are uploaded.
- param.op: indicates the operation number and completion status when the message type is GATTC_MSG_CMP_EVT. Note: The corresponding operation is GATT_OP_PEER_SVC_REGISTERED. param.op.arg indicates the handle number of the scanned attribute. Other operations are meaningless.

2.5.4.5. Write To The Server

Taking the client profile of the code in SDK_folder\examples\none_evm\ble_simple_central\code\ble_simple_central.c as an example, the interaction process of the client for writing operations is shown in the following figure.

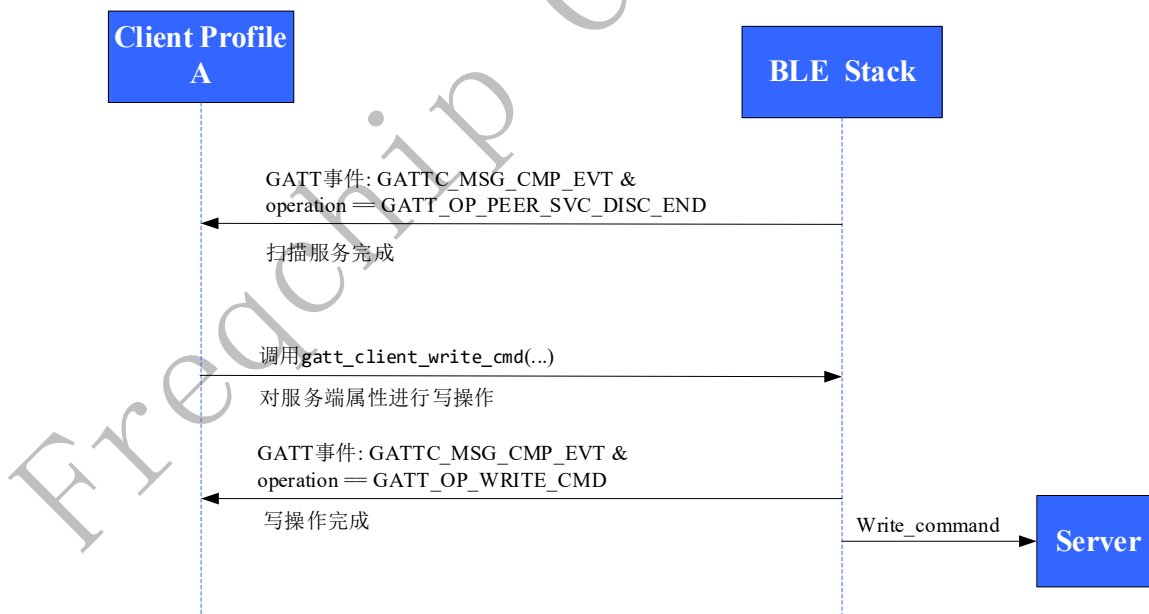


Figure 2-9 Client write_command Processing Flow

Description:

- After receiving the callback message from the client profile indicating that the scanning of the peer service is completed, according to the content of the GATT callback function in the previous section, it is necessary to check whether the handle number corresponding to the client uuid array is non - zero. Only the attributes corresponding to non - zero uuid can be subjected to write operations.
- After Step 1, the application layer can call the GATT write function to perform write operations on writable attributes. The write functions are defined in `gatt_api.h`, and there are three types in total as follows:
 - a) `gatt_client_write_req` represents a write operation, and the server is required to reply with a response.
 - b) `gatt_client_write_cmd` represents a write operation, and the server is not required to reply with a response.
 - c) `gatt_client_enable_ntf` represents enabling the ntf function of a certain attribute on the server side.
- When calling the `write_cmd` operation to send data to the protocol stack, the protocol stack will allocate a block of memory to cache the data sent by the application layer, and then immediately upload the write completion message to the client callback function. The message is `GATTC_MSG_CMP_EVT` & operation == `GATT_OP_WRITE_REQ` or `GATT_OP_WRITE_CMD`. Users can judge this message in the callback function as the processing for the completion of `write_cmd` data sending.
- When the application layer data is sent by the user through calling `write_req` and `enable_ntf`, the operation completion message will be uploaded only after the peer receives it successfully. Therefore, before the previous operation is completed, the application layer can only call `write_req` and `enable_ntf` operations once for the same attribute. Otherwise, subsequent `write_req` and `enable_ntf` operations will be invalid.
- When the application layer data is sent by the user through calling `write_cmd` (not `write_req`) to send data, the protocol stack only caches the data internally and sends

the cached data to the peer BLE device when it can be sent at the link layer.

Therefore, when the application layer sends data to the protocol stack, it can call the write operation multiple times. However, attention should be paid to the consumption of system memory. Generally, it is recommended to set a maximum number of data packets that can be sent to the protocol stack, use a counter to record the current number of packets sent, increment the counter by 1 each time a packet is sent successfully, and decrement it by 1 each time the write function is called to send data. The sample code is as follows:

```
uint8_t nb_report = 0;
uint16_t client_msg_handler(gatt_msg_t *p_msg)
{
    switch(p_msg->msg_evt)
    {
        case GATTC_MSG_CMP_EVT:
        {
            if(p_msg->param.op.operation == GATT_OP_WRITE_CMD )
            {
                nb_report--;
                write_att_cmd(p_msg->conidx, client_id, p_data, data_len );
            }
            break;
        }
    }
}

void write_att_cmd(uint8_t conidx, uint8_t client_id, uint8_t *p_data, uint16_t data_len )
{
    if(nb_report < 20)
    {
        gatt_client_write_t write;
        write.conidx = conidx;
        write.client_id = client_id;
        write.att_idx =1; //RX
        write.p_data = p_data;
        write.data_len =data_len;
        gatt_client_write_cmd(write);
        nb_report++;
    }
}
```

2.5.4.6. Read Operation To The Server

Take the client profile whose code is in SDK_Folder\examples\none_evm\ble_simple_central\code\ ble_simple_central.c as an example. The following figure shows the interaction process of the client read operation.

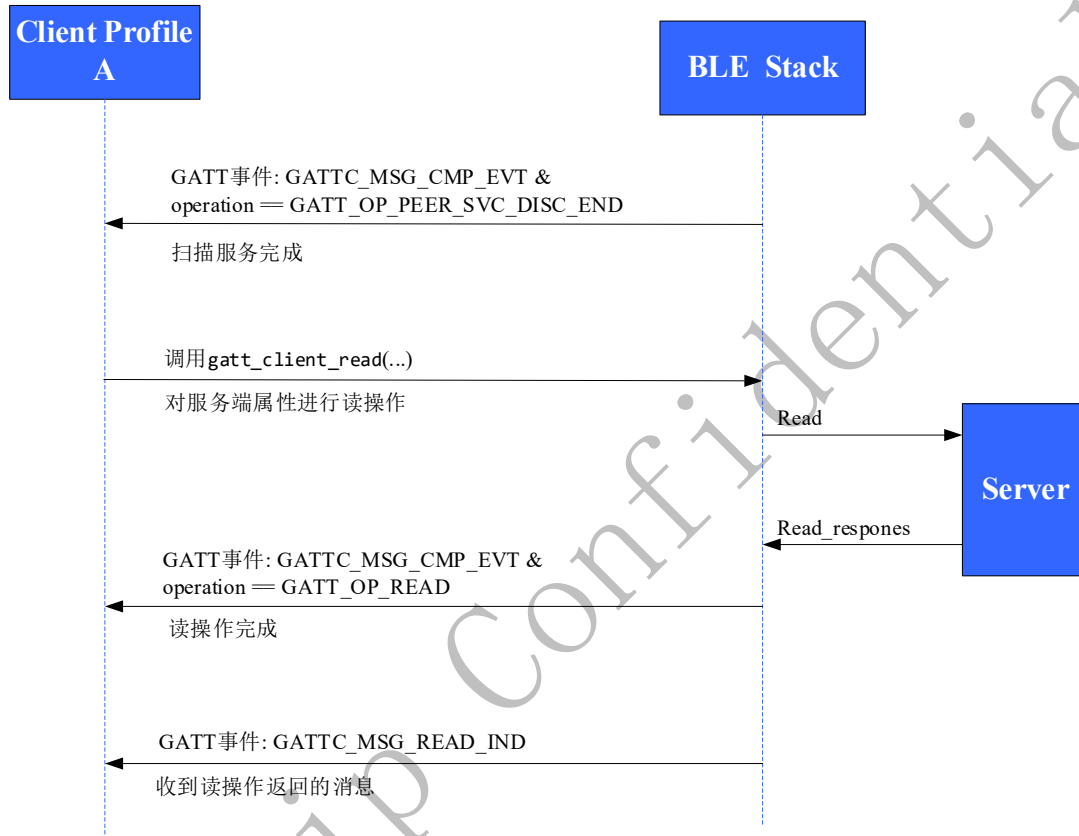


Figure 2-10 Client read processing flow

Note:

- After receiving the callback message from the client profile indicating that the scanning of the peer service is completed, according to the content of the GATT callback function in the previous section, it is necessary to check whether the handle number corresponding to the client's uuid array is not 0. Only the attributes corresponding to the uuid with a non-zero value can be subjected to a read operation.
- After step 1, the application layer may call a GATT write function to perform a write operation on the writable attribute. The read function is `gatt_client_read`, which is defined in `gatt_api.h`.

- After calling the read function to initiate a read operation, the message indicating the completion of the operation will not be uploaded until the peer successfully receives the operation and then replies with a read_response. The message is `GATTC_MSG_CMP_EVT & operation == GATT_OP_READ`. Users can determine this message within the callback function as the completion of the read operation.
- After the message reporting the completion of the Read operation is uploaded, the protocol stack will upload the GATT message `GATTC_MSG_READ_IND` again. This message indicates that the data replied by the peer has been received.

2.5.5. Basic Process of GATT Server

2.5.5.1. Server

The GATT server is responsible for receiving commands and requests sent by the peer device GATT client, and responding to the peer device according to the received commands and requests, or sending notifications or instructions to the peer device. The function of GATT server needs to work closely with various profile to complete.

The following section describes how to add profile and register profile callback functions in the GATT server, and how the GATT server handles client read/write requests. API for a detailed description, please refer to `g_att_api.h` function definition description. The basic process of server GATT is shown in the following figure.

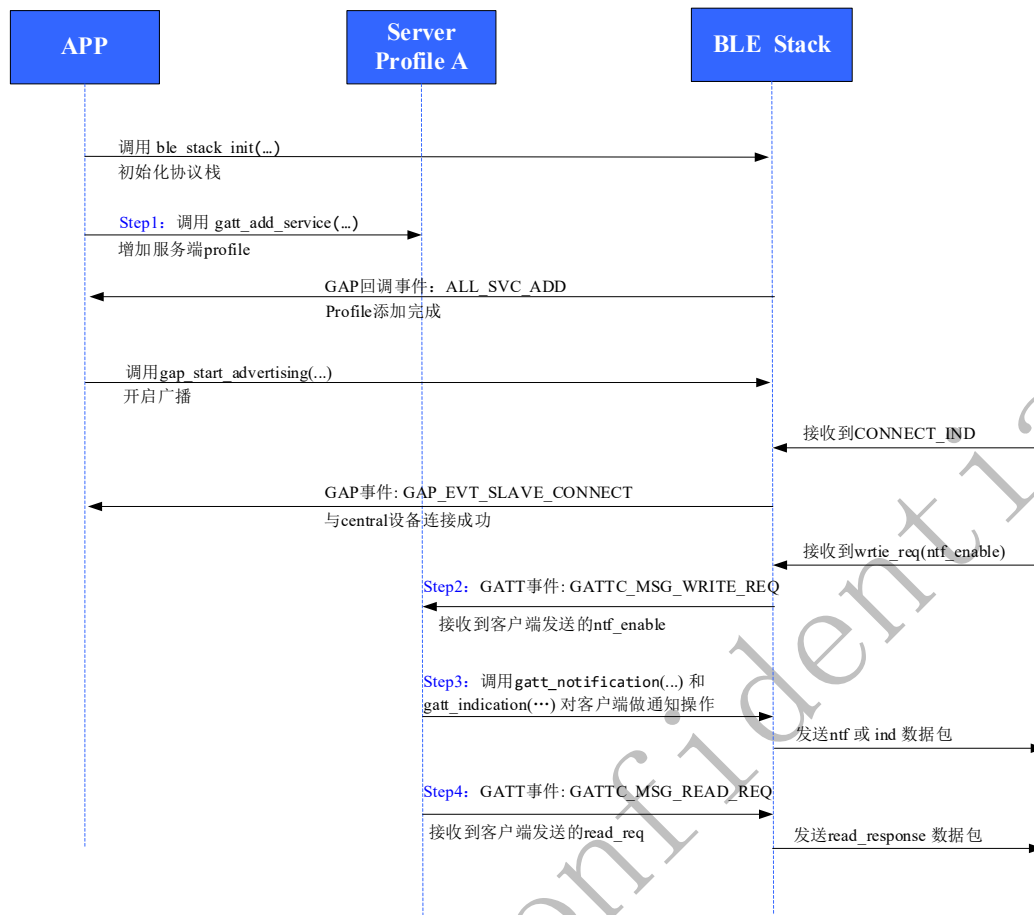


Figure 2-11 Server GATT Basic Process

2.5.5.2. Add server profile

For the server profile, the user needs to provide an attribute array att_table and the callback function of the profile, and register a server profile, so that the protocol stack can upload messages and listen to the messages of specific attributes. The following uses an example of registering a server profile to explain this process.

```

uint8_t svc_id = 0;
const uint8_t svc_group_uuid[] =
    "\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f";
const gatt_attribute_t att_db[] =
{
    [0] = { {UUID_SIZE_2, UUID16_ARR(GATT_PRIMARY_SERVICE_UUID)}
        , GATT_PROP_READ, UUID_SIZE_16, (uint8_t *)svc_group_uuid
    },
    [1] = { {UUID_SIZE_2, UUID16_ARR(GATT_CHARACTER_UUID)}
        , GATT_PROP_READ, 0, NULL
    },
    [2] = { {UUID_SIZE_16, {0x00, 0x00, 0x48, 0x43, 0x45, 0x54, 0x43, 0x49, 0x47, 0x4F, 0x4C,
        0x49, 0x04, 0x18, 0x00, 0x00, 0x00}}
    }
}
    
```



```

    , GATT_PROP_READ/GATT_PROP_NOTI/GATT_PROP_IND, 20, NULL
  },
  [3] = { {UUID_SIZE_2, UUID16_ARR(GATT_CLIENT_CHAR_CFG_UUID)}
    , GATT_PROP_READ/GATT_PROP_WRITE, 0, NULL
  },
  [4] = { {UUID_SIZE_2, UUID16_ARR(GATT_CHAR_USER_DESC_UUID)}
    , GATT_PROP_READ/GATT_PROP_WRITE, 0xC, NULL
  },
};
uint16_t svc_msg_handler(gatt_msg_t *p_msg)
{
    switch(p_msg->msg_evt)
    {
        case GATTC_MSG_READ_REQ:
            break;
    }
}
void user_main(void)
{
    gatt_service_t service;
    service.p_att_tb = att_db;
    service.att_nb = 5;
    service.gatt_msg_handler = svc_msg_handler;
    svc_id=gatt_add_service(&service);
}

```

Instructions:

- att_tb[] is an attribute array of the server profile. For the definition of the attribute table, please refer to 2.5.3.1.
- In the first line, the svc_id variable records the ID number of this server profile. This value will be needed when performing ntf/ind operations on the client later.
- In the 22nd line, svc_msg_handler is the event callback function corresponding to the server profile. When the protocol stack needs to report the messages corresponding to the profile, this function will be called.
- Lines 23 to 27 are the codes for the callback function to handle the messages uploaded by the protocol stack.
- Lines 32 to 36 define the variables of the server profile. After assigning the attribute table array and the callback function to this variable, the gatt_add_service function is called to add this server profile to the protocol stack.
- During initialization, the user can repeatedly call the gatt_add_service function to add multiple server profiles to the protocol stack.

- After all profiles (including client and server profiles) are successfully added, the protocol stack will upload the GAP event: GAP_EVT_ALL_SVC_ADDED.
- Profiles can only be added during initialization. Once the GAP event GAP_EVT_ALL_SVC_ADDED is uploaded, the user cannot call the gatt_add_service function to add new profiles.

2.5.5.3. Waiting For The Client To Send The Ntf/Ind Enable Signal

After the link is established, the device with server profile is added. After receiving the notification/indication enable message sent by the client, the ntf/ind message can be sent to the client. See the following sample code:

```
uint16_t svc_msg_handler(gatt_msg_t *p_msg)
{
    switch(p_msg->msg_evt)
    {
        case GATTC_MSG_WRITE_REQ:
            if(p_msg->att_idx == 3)
            {
                if( *(uint8_t *) (p_msg->param.msg.p_msg_data) & 0x1 )
                    ntf_enable[p_msg->conn_idx] = true;
                if( *(uint8_t *) (p_msg->param.msg.p_msg_data) & 0x2 )
                    ind_enable[p_msg->conn_idx] = true;
            }
            break;
    }
}
```

Instruction:

- After the link is established, the entry for handling the event of the client writing data can be obtained by determining that the GATT event type is GATTC_MSG_WRITE_REQ. The enabling of ntf/ind can be judged by determining the bit setting of the first byte of the written data.
- For the GATT event callback processing corresponding to the server profile, see the next section.

2.5.5.4. Server Profile Callback Function

In order to handle the read and write operations of profile attributes, the profile needs to define its own read and write callback functions. The callback function receives all messages about this profile uploaded by the protocol stack. The following uses an example of the callback function of a server profile to explain.

```
uint16_t svc_msg_handler(gatt_msg_t *p_msg)
{
    switch(p_msg->msg_evt)
    {
        case GATT_MSG_READ_REQ:
            if(p_msg->att_idx == 4)
            {
                memcpy(p_msg->param.msg.p_msg_data, "\x7\x8\x9", 3);
                return 3;
            }
            break;
        case GATT_MSG_WRITE_REQ:
            if(p_msg->att_idx == 4)
            {
                show_reg(p_msg->param.msg.p_msg_data, p_msg->param.msg.msg_len, 1);
            }
            break;
        case GATT_MSG_CMP_EVT:
            if(p_msg->att_idx == GATT_OP_NOTIFY)
            {
                co_printf("ntf done\r\n");
            }
            break;
        case GATT_MSG_LINK_CREATE:
            co_printf("ntf done\r\n");
            break;
        case GATT_MSG_LINK_LOST:
            co_printf("ntf done\r\n");
            break;
        default:
            break;
    }
    return 0;
};
```

Instructions:

- A message variable will be passed to the GATT event callback function of the profile. This variable includes the message type, link number, the serial number of the uuid array corresponding to the message, the handle number corresponding to the message, and the content of the message.
- The GATT event callback function processes the messages according to their types respectively, and the function ends with "return 0".

- When the message is `GATTC_MSG_READ_REQ`, it means a read message sent from the server device has been received. In the example, a response is made. Note: After the response is completed, the length of the response data needs to be returned instead of "return 0".
- When the message is `GATTC_MSG_WRITE_REQ`, it means a write message sent from the server device has been received. In the example, it is printed.
- When the message is `GATTC_MSG_CMP_EVT`, it means that a certain operation on the server profile is completed. `p_msg->param.op.operation` represents the type of the operation. There are the following types in total. Please refer to the macro - definition group `GATT_OPERATION_NAME` in `gatt_api.h`.

<code>#define GATT_OP_NOTIFY</code>	<code>0x01</code>	<code>///< GATT notification operation</code>
<code>#define GATT_OP_INDICA</code>	<code>0x02</code>	<code>///< GATT indication operation</code>
<code>#define GATT_OP_PEER_SVC_REGISTERED</code>	<code>0x03</code>	<code>///< Used with GATTC_CMP_EVT, GATT peer device service registered</code>
<code>#define GATT_OP_WRITE_REQ</code>	<code>0x05</code>	<code>///< GATT write request operation</code>
<code>#define GATT_OP_WRITE_CMD</code>	<code>0x06</code>	<code>///< GATT write command operation, write without response</code>
<code>#define GATT_OP_READ</code>	<code>0x07</code>	<code>///< GATT read operation</code>
<code>#define GATT_OP_PEER_SVC_DISC_END</code>	<code>0x08</code>	<code>///< Used with GATTC_CMP_EVT, GATT peer device service discovery is ended</code>

For server profile, the available operations are 0x01 and 0x02.

- 0x01 indicates that the notification operation (no response required) on the client attributes is completed.
- 0x02 indicates that the indication operation (response required) on the client attributes is completed.

The type of the parameter for the server profile callback function is `gatt_msg_t`. It is the same as the explanation of the input parameter for the client profile. Refer to Section 2.5.4.4.

2.5.5.5. Ntf/ind Operation To Client

the interaction process of performing ntf/ind operations to the client profile is shown in Figure 2-12.

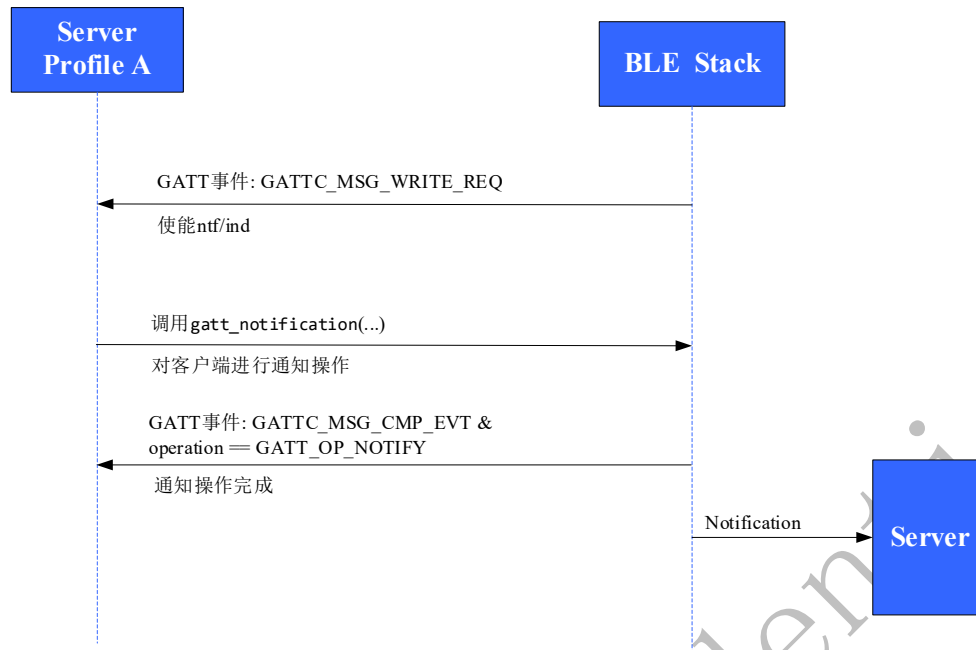


Figure 2-12 Server notification Processing Flow

Instructions:

- After receiving the GATT message GATTC_MSG_WRITE_REQ uploaded by the server profile, determine that if bit 0 of the first byte of the written data is set, the ntf (notification) is enabled. Determine that if bit 1 of the first byte of the written data is set, the ind (indication) is enabled.
- After the NTF is enabled, the application layer may call a GATT write function to perform a notification operation on the attribute of the enabled NTF. Notification functions are defined in gatt_api.h, and there are two types as follows:
 - a) gatt_notification indicates a notification operation and does not require the server to reply to the response.
 - b) Indicates the instruction operation, and a response from the server is required.
- When calling the notification operation to send data to the protocol stack, the protocol stack will allocate a block of memory to cache the data sent by the application layer, and then immediately upload the ntf completion message to the server callback function. The message is GATTC_MSG_CMP_EVT & operation == GATT_OP_NOTIFY. The user can determine this message in the callback function to handle it as the completion of the notification data sending.

- When calling the indication operation to send data to the protocol stack, the completion message of the operation will not be uploaded until the peer end successfully receives it. Therefore, before the previous operation is completed, the application layer can only call the indication operation once for the same attribute. Otherwise, the subsequent indication operations will be invalid.
- When the application layer sends data by calling the notification function, the protocol stack only caches the data internally and will send the cached data to the peer BLE device when the link layer is ready to send. So when the application layer sends data to the protocol stack, it can call the write operation multiple times repeatedly. However, attention should be paid to the consumption of system memory. It is generally recommended to set a maximum number of data packets that can be sent to the protocol stack, and use a counter to record the current number of sent packets. The counter is incremented by 1 each time a packet is successfully sent, and decremented by 1 each time the write function is called to send data. The sample code is as follows:

```

uint8_t nb_report = 0;
uint16_t svc_msg_handler (gatt_msg_t *p_msg)
{
    switch(p_msg->msg_evt)
    {
        case GATTC_MSG_CMP_EVT:
        {
            if(p_msg->param.op.operation == GATT_OP_NOTIFY )
            {
                nb_report--;
                att_ntf(p_msg-> conn_idx, svc_id, p_data, data_len );
            }
            break;
        }
    }
}

void att_ntf( uint8_t conidx, uint8_t svc_id, uint8_t *p_data, uint16_t data_len )
{
    if(nb_report < 20 && ntf_enable[conidx])
    {
        gatt_ntf_t ntf_att;
        ntf_att.conidx = conidx;
        ntf_att.svc_id = svc_id;
        ntf_att.att_idx = 2;
        ntf_att.p_data = p_data;
        ntf_att.data_len = data_len;
        gatt_notification (ntf_att);
        nb_report++;
    }
}
  
```

2.5.5.6. Respond to a read operation to the client

After the server receives the read request, the interaction process of replying to the data to the client is shown in Figure 2-13.

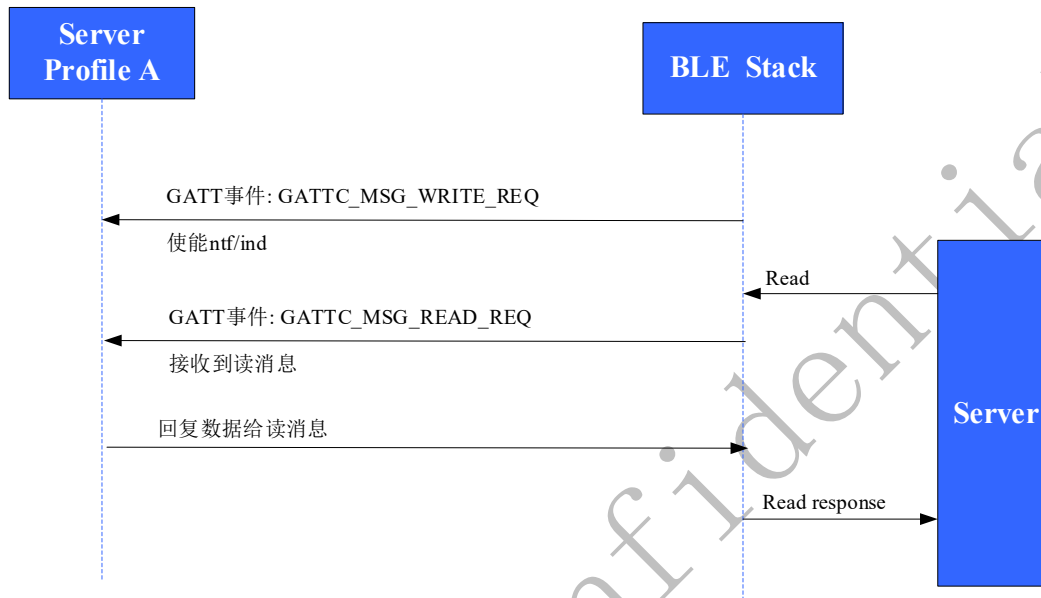


Figure 2-13 Server Read Response Processing Flow

Instructions:

- After receiving the GATT message GATTC_MSG_WRITE_REQ uploaded by the server profile and judging that ntf/ind is enabled, it is confirmed that the client has correctly scanned the attribute handle number of the server.
- When receiving the GATT message GATTC_MSG_READ_REQ uploaded by the server profile, it indicates that the client needs to read the data of a certain attribute. The server needs to respond to this read operation, as shown in the following code.

```

uint16_t svc_msg_handler(gatt_msg_t *p_msg)
{
    switch(p_msg->msg_evt)
    {
        case GATTC_MSG_READ_REQ:
            if(p_msg->att_idx == 2)
            {
                memcpy(p_msg->param.msg.p_msg_data, "\x7\x8\x9", 3);
                return 3;
            }
            break;
    }
}

```

First, determine that the serial number of the attribute targeted by the read message in the attribute table is 2. Then, directly copy the read data to be replied to the buffer pointed to by the data pointer of the callback function parameter. Finally, just return the length of the copied data. After the callback function returns, the protocol stack will send a read response packet to the client.

2.6. Safety Management

2.6.1. Pairing

The pairing process, that is, the process of generating and distributing keys, can be divided into three stages:

1. Exchange pairing information.
2. Generate a link encryption key.
3. Distribute other specified key information over the encrypted link, and decide whether to store the distributed key information in the security database according to whether the device supports binding.

The pairing flow chart is as follows. The pairing methods are divided into two types: LE Legacy Pairing and LE Secure Connection (added from the core 4.2 version). The main difference between the two lies in step 2 - the process of generating keys. In addition, in this step, LE Legacy Pairing generates STK, and LE Secure Connection generates LTK.

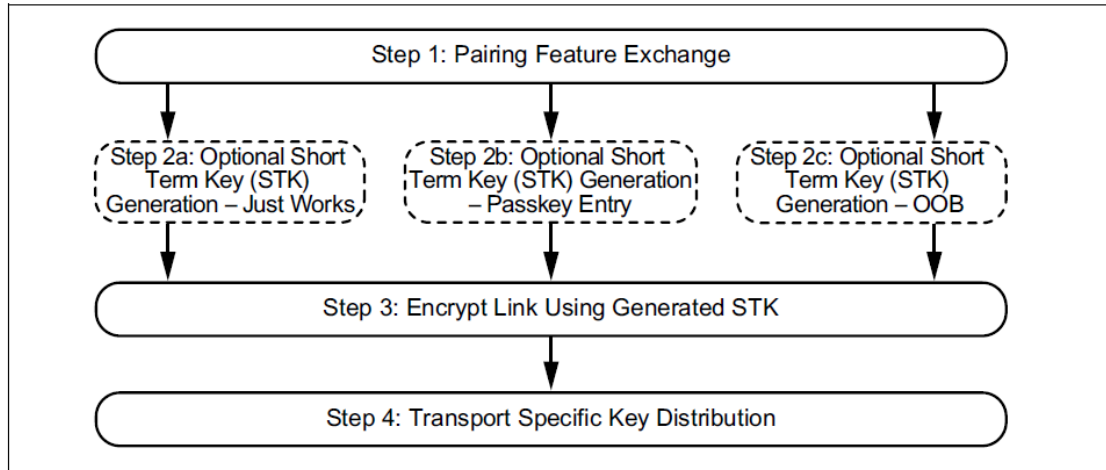


Figure 2-14 BLE Pairing Process

In LE Secure Connection, the Elliptic Curve Diffie-Hellman encryption algorithm is introduced, which greatly improves the security. According to the security features supported by the device, the pairing methods can be divided into the following four types:

- Just Works (Secure Connections or Legacy) is suitable for the situation where neither of the two parties in the pairing has input/output IO devices (such as keyboard input or display output). Since there is no need for MITM (Man-in-the-Middle) authentication during the Just Works pairing process, it cannot resist MITM attacks. Just Works pairing can be used for both Legacy Pairing and Secure Connection.
- Passkey Entry (Secure Connections or Legacy): The pairing process supports MITM (Man-in-the-Middle) authentication and can be used for both Legacy Pairing and Secure Connection. During the pairing process, the user is required to enter a 6 - digit decimal password, which serves as the input parameter for the pairing process.
- Numeric Comparison(Secure Connections). If both devices support Secure Connection, the IO capability has display and input functions, and MITM authentication is required, the Numeric Comparison pairing process can be used.
- Out of Band(Secure Connections or Legacy), this method interacts with the intermediate value required for the pairing process through out-of-band methods such as NFC. This pairing method is not supported in the current version of the SDK.

2.6.1.1. JustWorks Pairing

If neither of the two devices requires Man - in - the - Middle (MITM) authentication, the Just Works pairing process can be used. Since there is no need for MITM authentication during the Just Works pairing process, it cannot resist MITM attacks. Just Works pairing can be either Legacy Pairing or Secure Connection. Users only need to configure the security parameters before the pairing process starts. Once the pairing process is initiated, no user interaction is required during the pairing.

The following takes the Legacy Pairing and Just Works pairing method as an example to introduce how to configure the parameters:

```
gap_security_param_t param =
{
    .mitm = false,
    .ble_secure_conn = false,
    .io_cap = GAP_IO_CAP_NO_INPUT_NO_OUTPUT,
    .pair_init_mode = GAP_PAIRING_MODE_WAIT_FOR_REQ,
    .bond_auth = true,
    .password = 0,
};
gap_security_param_init(& param);
```

Instructions:

- The mitm parameter indicates whether to enable man - in - the - middle authentication. Setting it to false means not to enable it.
- The ble_secure_conn parameter indicates whether to enable the security connection. Setting it to false means not to enable it.
- io_cap indicates the io output capability of the device, whether it has the capability of displaying or keyboard input. There is no io capability in this example
- The pair_init_mode parameter indicates whether pairing is enabled. In the example, pairing is enabled.

- `bond_auth` parameter indicates whether binding is enabled, that is, whether key distribution is performed after pairing is completed for the next connection. Setting it to true means enabling bonding.
- The `password` parameter is used to set the authentication code for man - in - the - middle (MITM) authentication. The peer device needs to enter this authentication code for authentication. It is only useful when the `io_cap` includes output capabilities and the `mitm` is enabled.

2.6.1.2. Passkey Entry Pairing

The Passkey Entry pairing process supports MITM (Man-in-the-Middle) authentication.

The pairing can be either Legacy Pairing or Secure Connection. During the pairing process, the device with display output capabilities shows a 6-digit decimal password, and the device with keyboard input capabilities enters this password as the temp key to proceed with the subsequent pairing process.

The following introduces the parameter settings for the Passkey Entry pairing mode with a configuration for a device having output capabilities:

```
gap_security_param_t param =
{
    .mitm = true,
    .ble_secure_conn = false,
    .io_cap = GAP_IO_CAP_DISPLAY_ONLY,
    .pair_init_mode = GAP_PAIRING_MODE_WAIT_FOR_REQ,
    .bond_auth = true,
    .password = 123456,
};
gap_security_param_init(& param);
```

Instructions:

- The `mitm` parameter indicates whether to enable man - in - the - middle authentication. Setting it to true means enabling it.
- The `ble_secure_conn` parameter indicates whether to enable the security connection. Setting it to false means not to enable it.

- The `io_cap` parameter indicates the IO output capabilities of the device, i.e., whether it has display or keyboard input capabilities. In this example, the device has display capabilities and can prompt the peer device to enter the authentication code.
- The `pair_init_mode` parameter indicates whether pairing is enabled. In the example, pairing is enabled.
- The `bond_auth` parameter indicates whether binding is enabled. Setting to true means enable binding.
- The `password` parameter is used to set the authentication code for man-in-the-middle (MITM) authentication. The peer device needs to enter this authentication code for authentication.

For a device with input capabilities, the parameter settings for the Passkey Entry pairing mode are as follows:

```
gap_security_param_t param =
{
    .mitm = true,
    .ble_secure_conn = false,
    .io_cap = GAP_IO_CAP_KEYBOARD_ONLY,
    .pair_init_mode = GAP_PAIRING_MODE_WAIT_FOR_REQ,
    .bond_auth = true,
    .password = 123456,
};
gap_security_param_init(& param);
```

When the password to be entered is displayed on the peer device, the following function can be called to pass the password to the protocol stack for the subsequent pairing process:

```
void gap_security_send_pairing_password(uint32_t conidx, uint32_t value)
```

Instructions:

- `conidx` is the index of the connection.
- `value` is the password.

2.6.1.3. Security Connections Pairing

If both devices support Secure Connection pairing, you can use Secure Connection pairing process. The specific security parameter configuration can be referred to as follows:

```
gap_security_param_t param =
{
    .mitm = false,
    .ble_secure_conn = true,
    .io_cap = GAP_IO_CAP_NO_INPUT_NO_OUTPUT,
    .pair_init_mode = GAP_PAIRING_MODE_WAIT_FOR_REQ,
    .bond_auth = true,
    .password = 0,
};
gap_security_param_init(& param);
```

Instructions:

- The mitm parameter indicates whether to enable authentication with middleman, and is set to false to indicate that it is not enabled.
- The ble_secure_conn parameter indicates whether to start the security connection. Setting to true means enabling it.
- io_cap indicates the I/O capability of this device, whether it has the capability of display or keyboard input. There is no io capability in this example.
- The pair_init_mode parameter indicates whether pairing is enabled. In the example, pairing is enabled.
- The bond_auth parameter indicates whether authentication is enabled. Setting it to true means enabling authentication.
- The password parameter is used to set the authentication code for MITM (Man - in - the - Middle) authentication. The peer device needs to input this authentication code for authentication. It is only useful when the io_cap includes output capabilities and mitm is enabled.

2.6.1.4. Pairing prohibition

The parameter settings for prohibiting pairing are as follows:

```
gap_security_param_t param =
{
    .mitm = false,
    .ble_secure_conn = false,
    .io_cap = GAP_IO_CAP_NO_INPUT_NO_OUTPUT,
    .pair_init_mode = GAP_PAIRING_MODE_NO_PAIRING,
    .bond_auth = false,
    .password = 0,
};
gap_security_param_init(& param);
```

Instructions:

- The `pair_init_mode` parameter indicates whether to enable pairing. In the example, it is set to "no pairing" to prohibit pairing.
- The `bond_auth` parameter indicates whether binding is enabled. Setting it to false means not enabling bonding.

2.6.1.5. Saving Of Pairing Keys

If the user program needs to save information such as bonding information and peer service information, the bonding management initialization function needs to be called before the protocol stack starts running.

```
void gap_bond_manager_init(uint32_t flash_addr, uint32_t svc_flash_addr, uint8_t max_dev_num, bool enable);
```

Instructions:

- The function input parameter `flash_addr` indicates the flash address used to store the key generated during pairing. It must be an integer multiple of 0x 1000. A sector(4KB) is reserved in the SDK for storing related content.
- The `svc_flash_addr` indicates the flash storage address of the server profile when the device is used as a client. It must be an integer multiple of 0x1000. A sector(4KB) is reserved in the SDK for storing related content.
- `max_dev_num` indicates the maximum number of bound devices supported by the bonding management function. Value range is [1,20]. Each device with a different MAC address counts as one. If the number of stored devices exceeds this value, the storage information of the 0th device will be overwritten starting from the beginning.
- The `enable` parameter indicates whether to store pairing information. If set to true, the protocol stack will automatically store the keys generated during pairing to the flash. If set to false, the protocol stack will not store the keys generated during pairing.

- When this function is called during the initialization of the entire project, all the stored keys and service information will be read from the flash address specified by the input parameters into internal variables.

2.6.1.6. Host Initiated Pairing

After the host initiates the pairing operation, the interaction between the application layer and the BLE protocol stack is shown in the following figure.

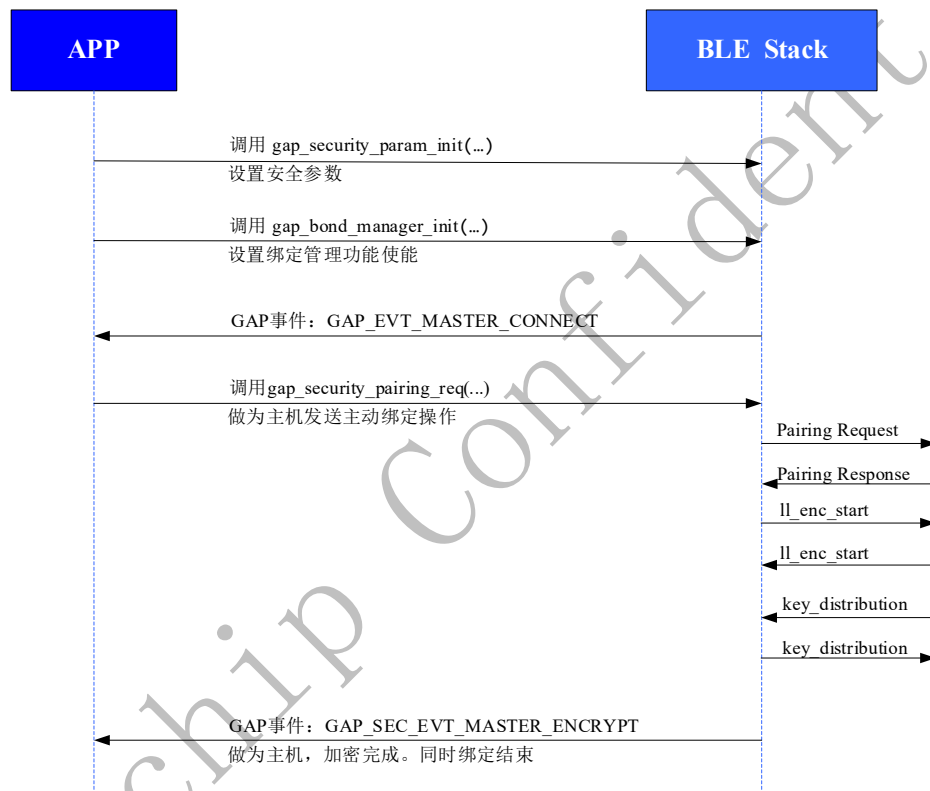


Figure 2-15 Host Initiates Pairing Operation

The specific steps for the host to open the pairing operation process are as follows:

```

void proj_ble_gap_evt_func(gap_event_t *event)
{
    switch(event->type)
    {
        case GAP_EVT_MASTER_CONNECT:
        {
            if (gap_security_get_bond_status())
                gap_security_enc_req(p_event->param.master_connect.conidx);
            else
                gap_security_pairing_req(p_event->param.master_connect.conidx);
        }
        break;
        case GAP_SEC_EVT_MASTER_ENCRYPT:
            co_printf("master[%d]_encrypted\r\n", event->param.master_encrypt_conidx);
    }
}
    
```

```

        break;
    }
}

```

Instructions:

- In the GAP event callback function, determine whether the event type is GAP_EVT_MASTER_CONNECT to obtain the entry for handling the host connection event. In this event, call gap_security_get_bond_status() to check whether the peer device of the current connection is a bonded device. If it is an unbonded device, call the gap_security_pairing_req() function to initiate the bonding action.
- After bonding, the connection will be automatically encrypted. After the encryption is successful, the BLE protocol stack will upload the GAP event GAP_SEC_EVT_MASTER_ENCRYPT.

2.6.1.7. Pairing from Slave

When the slave needs to initiate a pairing request, the interaction between the application layer and the BLE protocol stack is shown in the following figure

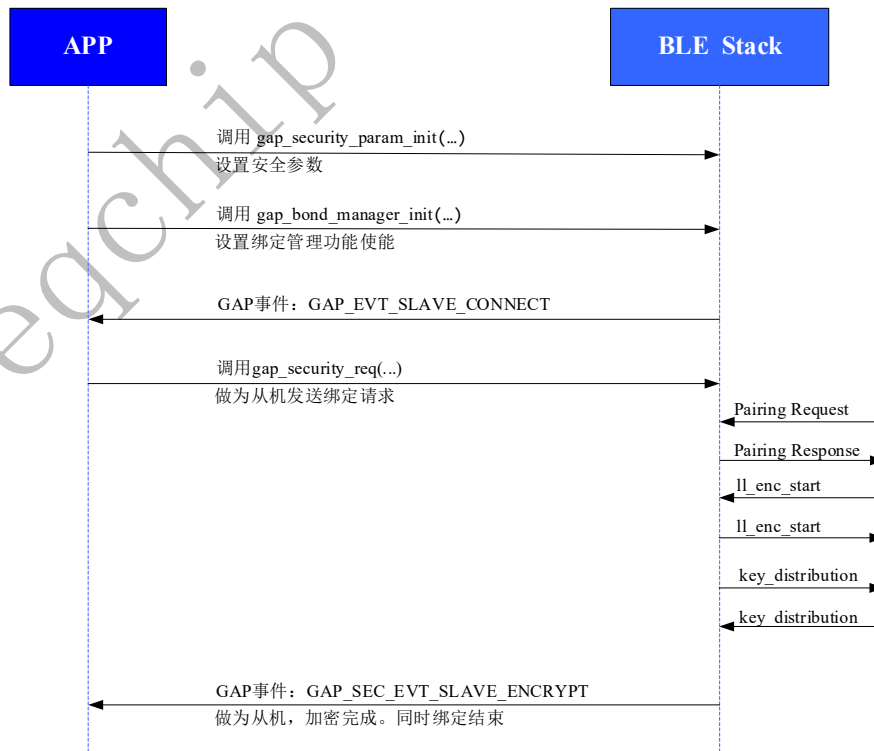


Figure 2-16 Slave Initiates Pairing Operation

The specific steps for the slave to initiate the pairing request process are as follows:

```
void proj_ble_gap_evt_func(gap_event_t *event)
{
    switch(event->type)
    {
        case GAP_EVT_SLAVE_CONNECT:
        {
            gap_security_req(event->param.slave_connect.conidx);
        }
        break;
        case GAP_SEC_EVT_SLAVE_ENCRYPT:
            co_printf("slave[%d]_encrypted\r\n", event->param.slave_encrypt_conidx);
            break;
    }
}
```

Instructions:

- In the GAP event callback function, obtain the entry for handling the slave connection event by determining that the event type is GAP_EVT_SLAVE_CONNECT. Under this event, call the gap_security_req function to request the host to initiate the bonding action.
- After bonding, the connection will be automatically encrypted. Once the encryption is successful, the BLE protocol stack will upload the GAP event GAP_SEC_EVT_SLAVE_ENCRYPT.

2.6.2. Encryption

After the device has completed the pairing process, as long as the bonding management function is enabled, the encryption function can be directly called to perform the encryption operation on the established connection. The encryption operation is initiated by the host after the connection is established.

2.6.2.1. Host Initiated Encryption

```
void proj_ble_gap_evt_func(gap_event_t *event)
{
    switch(event->type)
    {
        case GAP_EVT_MASTER_CONNECT:
        {
            if (gap_security_get_bond_status())
                gap_security_enc_req(event->param.master_connect.conidx);
        }
    }
}
```

```

        else
            gap_security_pairing_req(p_event->param.master_connect.conidx);
        }
        break;
    case GAP_SEC_EVT_MASTER_ENCRYPT:
        co_printf("master[%d]_encrypted\r\n", event->param.master_encrypt_conidx);
        break;
    }
}

```

Instructions:

- In the GAP event callback function, determine the event type as GAP_EVT_MASTER_CONNECT to get the entry point for handling the host connection event. Under this event, call gap_security_get_bond_status() to check whether the peer device of the current connection is a bonded device. If it is a bonded device, call the gap_security_enc_req() function to initiate the encryption action.
- After the encryption is successful, the BLE protocol stack uploads the GAP event GAP_SEC_EVT_MASTER_ENCRYPT.

2.6.2.2. Slave Response To Encryption

```

void proj_ble_gap_evt_func(gap_event_t *event)
{
    switch(event->type)
    {
        case GAP_EVT_SLAVE_CONNECT:
        {
            }
        break;
        case GAP_SEC_EVT_SLAVE_ENCRYPT:
            co_printf("slave[%d]_encrypted\r\n", event->param.slave_encrypt_conidx);
            break;
    }
}

```

Instructions:

- In the GAP event callback function, determine the event type as GAP_EVT_SLAVE_CONNECT to obtain the entry for handling the slave connection event.

- After the encryption is successful, the BLE protocol stack uploads the GAP event GAP_SEC_EVT_SLAVE_ENCRYPT.

2.6.3. Privacy Management

Privacy management in BLE means that authenticated devices can track and identify the target device, while other non - authenticated devices cannot. Privacy management enables authenticated devices to connect and communicate with the target device normally, and at the same time prevents other non - authenticated devices and malicious devices from tracking the target device.

2.6.3.1. Enabling the Privacy Management Function

A whitelist can be set during system initialization, which can filter out devices outside the whitelist during broadcasting and active connection. The specific steps to enable the whitelist management function are as follows:

1. The master device connects and bonds with the slave device. During the bonding process, the IRK and identity address information need to be exchanged. After bonding, the master device disconnects from the slave device. For the specific bonding process, please refer to section 2.6.1.
2. Whitelist configuration. Users can set the whitelist during the system initialization process, before sending broadcasts or initiating active connections.

```
gap_bond_manager_init(BLE_BONDING_INFO_SAVE_ADDR, BLE_REMOTE_SERVICE_SAVE_ADDR, 1, true);
gap_bond_info_t info;
gap_bond_manager_get_info(0, &info);
if(info.bond_flag)
{
    gap_mac_addr_t mac_set[1];
    memcpy(&mac_set[0], &info.peer_addr, sizeof(gap_mac_addr_t));
    gap_set_wl(&mac_set[0], 1);
    gap_ral_t ral_set[1];
    memcpy(&ral_set[0].addr, &info.peer_addr, sizeof(gap_mac_addr_t));
    memcpy(ral_set[0].peer_irk, info.peer_irk, 16);
    gap_set_ral(&ral_set[0], 1);
}
```

Instructions:

- In the first line, call the binding management initialization function `gap_bond_manager_init` enable the binding management function.
- In the second and third lines, the call `gap_bond_manager_get_info` obtains the bonding information of the bonding device with serial number 0, including mac address and IRK.
- Line 4, when judging that the bonding status of the bonding device is true, set it to the white list
- In the sixth, seventh, and eighth lines, copy the mac address and type of the bonded device, and call `gap_set_wl` to set the mac address and type of the device to the whitelist.
- From the ninth to the twelfth lines, copy the IRK value of the bonded device and call the `gap_set_ral` function to set the device's IRK value to ral (resolved address list).
- If the address of the bonded device is of a non - resolved address type, i.e., a public or fixed private address, there is no need to execute the content from the ninth to the twelfth lines to set the RAL (Resolved Address List).

3. The broadcast device uses the whitelist. The whitelist filtering function can be enabled for broadcasting by setting the filtering policy in the broadcast parameters.

```
gap_adv_param_t adv_param;
adv_param.adv_filt_policy = GAP_ADV_ALLOW_SCAN_WLST_CON_WLST;
adv_param.adv_mode = GAP_ADV_MODE_UNDIRECT;
adv_param.disc_mode = GAP_ADV_DISC_MODE_GEN_DISC;
adv_param.adv_addr_type = GAP_ADDR_TYPE_PUBLIC;
adv_param.adv_chnl_map = GAP_ADV_CHAN_ALL;
adv_param.adv_intv_min = 0x40;
adv_param.adv_intv_max = 0x40;
gap_set_advertising_param(&adv_param);
```

Instructions:

- For the specific broadcast initiation process, see chapter 2.2.
- In the example code, the second line, the broadcast filtering option is set so that the device only responds to devices in the whitelist during scanning and connection.

After this setting, only the devices in the whitelist from Step 2 can scan and connect to this broadcast.

4. Use the whitelist for active connection. Users can call the function to initiate an active connection targeting whitelist devices, so as to connect only to the devices within the whitelist.

```
mac_addr_t peer_addr;
memcpy(peer_addr.addr, "\x00\x00\x00\x00\x00\x00", 6);
gap_start_conn_whitelist(&peer_addr, 0, 6, 6, 0, 400);
```

Instructions:

- Call the function `gap_start_conn_whitelist` to automatically connect to devices in the white list. This connection will not stop automatically, even if a BLE device is connected. Unless the user calls a `gap_stop_conn` function to stop the action.
- Before using the whitelist connection function, you must perform step 2 to set the whitelist.

2.6.3.2. Address Configuration Instructions

When enable broadcasting, scanning, and establishing a connection, the controller will use the local address set by the user to send air - interface packets. The local address only needs to be set once during initialization. The local address set during initialization is used as the device address during broadcasting, scanning and active connection.

1. BLE mac address type, there are four types of local address, please refer to the definitions in `ble_stack.h`.

```
enum ble_addr_type
{
  //addr is set by user, value is fixed forever
  BLE_ADDR_TYPE_PUBLIC,
  //addr is set by user, value is fixed during power on lifetime
  BLE_ADDR_TYPE_PRIVATE,
  //addr is generated by stack with IRK
  BLE_ADDR_TYPE_RANDOM_RESOVABLE,
  //addr is generated by stack and is randomly
  BLE_ADDR_TYPE_RANDOM_NONE_RESOVABLE,
};
```

Instructions:

- The PUBLIC type means that the device's MAC address will remain unchanged all the time. This type of address needs to be applied for from the IEEE.
 - The PRIVATE type means address can be specified by the user and remains unchanged within one power - on cycle or all the time. The highest 2 bits of this type of address should be set to 11b.
 - The RANDOM_RESOVABLE type address is generated through a random number and an IRK (Identity Resolving Key). Devices using this type of address can prevent being scanned and tracked by unknown devices.
 - The RANDOM_NONE_RESOVABLE type address is similar to the PRIVATE type address, but the RANDOM_NONE_RESOVABLE type address is updated periodically, and the update period is set by the GAP.
2. Setting the device's local address: During the initialization of user_main(), while assigning a value to the MAC address, the type of the local MAC address needs to be specified. Once the type of the local MAC address is set, it cannot be modified later. Scanning, broadcasting, and actively initiating a connection all use the local address type set at this time for operations.

```

/* set local BLE address */
mac_addr_t mac_addr;
mac_addr.addr[0] = 0xbd;
mac_addr.addr[1] = 0xad;
mac_addr.addr[2] = 0x10;
mac_addr.addr[3] = 0x11;
mac_addr.addr[4] = 0x20;
mac_addr.addr[5] = 0x20;
gap_address_set(&mac_addr, BLE_ADDR_TYPE_PRIVATE);
  
```

3. OSAL API

3.1. Task

The scheduling of this task depends on the non-preemptive polling scheduling mechanism of the main loop, where the events involved are executed in a first-in, first-out order. The creation of the task needs to be executed after the initialization of the BLE protocol stack, and the pushing of the message needs to be executed after the task is created.

3.1.1. Create a Task

Currently, the SDK supports users to create up to 20 tasks. `os_task_create` is used to create a new task. The parameter is the event callback function of the task. Since task scheduling depends on the main loop, the processing time of a single event should not be too long (tens of ms). The following code shows an example of creating a task:

```
static int user_task_func(os_event_t *param)
{
    switch(param->event_id)
    {
        case USER_EVT_AT_COMMAND:
            app_at_cmd_rcv_handler(param->param, param->param_len);
            break;
    }

    return EVT_CONSUMED;
}

void user_task_init(void)
{
    user_task_id = os_task_create(user_task_func);
}
```

3.1.2. push a Message to a Task

After the task is created, you can call the `os_msg_post` function to push messages to the task for asynchronous processing. The sample code is as follows:

```
os_event_t at_cmd_event;
at_cmd_event.event_id = USER_EVT_AT_COMMAND;
at_cmd_event.param = at_rcv_buffer;
```

```
at_cmd_event.param_len = at_rcv_index;
os_msg_post(user_task_id, &at_cmd_event);
```

The first parameter of the `os_msg_post` function is the id of the task that receives the message, and this ID is the return value when the task is created. The `event_id` in the message structure is used to identify the message type for the event callback function `user_task_func` to identify.

3.2. Timer

This timer is a software timer that depends on the operation of the baseband. Its accuracy is 10 ms. The current SDK supports creating a maximum of 40 timers. The callback function of the timer relies on the non - preemptive scheduling of the main loop, so the execution time in the callback function should not be too long (tens of milliseconds).

To create and enable a timer, the structure `test_timer` that represents the timer needs to reside in memory permanently, so local variables cannot be used. Before calling `os_timer_start`, you need to call `os_timer_init` for initialization. The third parameter in the `os_timer_start` function indicates whether this timer needs to be triggered cyclically

```
os_timer_init(&test_timer, test_timer_handler, NULL);
os_timer_start(&test_timer, 500, true);
```

In the callback function registered during initialization, scheduled tasks can be executed. The parameter of the callback function is the third parameter passed when calling `os_timer_init`:

```
void test_timer_handler(void *param)
{
    LOG_INFO(app_tag, "test timer\r\n");
}
```

Calling the `os_timer_stop` can stop a running timer:

```
os_timer_stop(&test_timer);
```


3.3. Memory

In the SDK, all unallocated RAM is reserved for the Heap space, and a set of functions are provided for users to call:

1. Allocate memory of a specified size

```
void *os_malloc(uint32_t size)
```

2. Reallocate memory of a specified size and retain the previously existing data

```
void *os_realloc(void *ptr, uint32_t new_size)
```

3. Free allocated memory

```
void os_free(void *ptr)
```

4. Get the currently available heap size

```
uint16_t os_get_free_heap_size(void)
```

4. Peripheral Driver

4.1. GPIO

The FR8000 series of chips have different models with different numbers of GPIOs in their packages. However, the use of GPIOs are the same. Each GPIO can be multiplexed into different functions and has functions such as external interrupt and wake - up. The control of GPIOs can be assigned to two modules: the normal control unit and the low - power control unit. When the system is in the normal operating mode, both control units can be used to control the GPIOs. When the system is in the low - power mode, the normal control unit is in the power - down state, and at this time, only the low - power control unit can control the GPIOs.

When the normal control unit controls the GPIOs, the GPIOs can be used as general - purpose input/output ports, external interrupt ports, serial ports, SPI ports and other common external interface functions. When the low - power unit controls the GPIOs, the GPIOs can be used as input/output ports, for external level change detection, Keyscan and other functions. The SDK provides two interfaces to set the control of GPIOs:

Set the GPIO control to the normal control unit:

```
void pmu_set_pin_to_CPU(enum system_port_t port, uint8_t bits);
```

Set GPIO control to low-power control unit:

```
void pmu_set_pin_to_PMU(enum system_port_t port, uint8_t bits);
```

4.1.1. GPIO Control (Normal Control Unit)

When GPIOs are controlled by the normal control unit, they can be multiplexed into functions of different peripherals. It should be noted that after entering the sleep state, the normal control unit will power down, causing these configurations to be lost. Therefore, after waking up from sleep, these configurations need to be re - called.

4.1.1.1. Input

The following example shows how to configure PA0 as an input function under the control of the normal control unit:

```
/* configure PA0 IO MUX */
system_set_port_mux(GPIO_PORT_A, GPIO_BIT_0, PORTA0_FUNC_A0);
/* set PA0 function to GPIO */
gpio_set_func(GPIO_PORT_A, GPIO_BIT_0, GPIO_FUNC_GPIO);
/* set PA0 to input mode */
gpio_set_dir(GPIO_PORT_A, GPIO_BIT_0, GPIO_DIR_IN);
/* get PA0 value */
gpio_get_pin_value(GPIO_PORT_A, GPIO_BIT_0);
/* get PORTA value */
gpio_porta_read();
```

4.1.1.2. Output

The following example shows how to configure PA1 as an output function under the control of the normal control unit:

```
/* configure PA1 IO MUX */
system_set_port_mux(GPIO_PORT_A, GPIO_BIT_1, PORTA1_FUNC_A1);
/* set PA1 function to GPIO */
gpio_set_func(GPIO_PORT_A, GPIO_BIT_1, GPIO_FUNC_GPIO);
/* set PA1 to input mode */
gpio_set_dir(GPIO_PORT_A, GPIO_BIT_1, GPIO_DIR_OUT);
/* get PA1 value */
gpio_set_pin_value(GPIO_PORT_A, GPIO_BIT_1, 1);
/* get PORTA value */
gpio_porta_write(gpio_porta_read() | 0x02);
```

4.1.1.3. Pull Up and Down

The following example shows how to configure pull - up/pull - down resistors and cancel their configuration:

```
/* set PA2 pull down */
system_set_port_pull(GPIO_PA2, GPIO_PULL_DOWN, true);
/* set PA3 pull up */
system_set_port_pull(GPIO_PA3, GPIO_PULL_UP, true);
/* remove PA4 pull down and pull up */
system_set_port_pull(GPIO_PA4, GPIO_PULL_NONE, false);
```

4.1.1.4. External Interrupt

The following example shows how to configure a low - level interrupt for PA1 with a anti-shake time of 10 ms:

```
/* configure PA1 IO MUX */
system_set_port_mux(GPIO_PORT_A, GPIO_BIT_1, PORTA1_FUNC_A1);
/* set PA1 function to GPIO */
gpio_set_func(GPIO_PORT_A, GPIO_BIT_1, GPIO_FUNC_EXTI);
/* set PA1 to input mode */
gpio_set_dir(GPIO_PORT_A, GPIO_BIT_1, GPIO_DIR_IN);
/* set PA1 pull up */
system_set_port_pull(GPIO_PA1, GPIO_PULL_UP, true);
/* set anti-shake time */
exti_set_control(EXTI_GPIOA_1, 1000, 10);
/* set trigger type to low level */
exti_set_type(EXTI_GPIOA_1, EXTI_TYPE_LOW);
/* enable PA1 external interrupt */
exti_enable(GPIO_PA1);
/* enable NVIC GPIO interrupt */
NVIC_EnableIRQ(GPIO_IRQn);
```

An example of the external interrupt handling function is as follows:

```
void exti_isr(void)
{
    uint32_t status;

    /* get and clear exti interrupt status */
    status = exti_get_status();
    exti_clear(status);

    if(status & GPIO_PA1) {
        /* reset trigger type to high level */
        exti_set_type(EXTI_GPIOA_1, EXTI_TYPE_HIGH);
    }
}
```

4.1.1.5. Configure IO for Other Functions

The following uses the I2C configuration as an example to show how to configure the IO as a functional pin for other peripherals:

```
system_set_port_pull(GPIO_PC4/GPIO_PC5/GPIO_PC2/GPIO_PC3, GPIO_PULL_UP, true);
system_set_port_mux(GPIO_PORT_C, GPIO_BIT_4, PORTC4_FUNC_I2C0_CLK);
system_set_port_mux(GPIO_PORT_C, GPIO_BIT_5, PORTC5_FUNC_I2C0_DAT);
system_set_port_mux(GPIO_PORT_C, GPIO_BIT_2, PORTC2_FUNC_I2C1_CLK);
system_set_port_mux(GPIO_PORT_C, GPIO_BIT_3, PORTC3_FUNC_I2C1_DAT);
```

4.1.2. GPIO Control (Low Power Control Unit)

When the GPIOs are controlled by the low-power control unit, the configured functions can work properly in both the normal operating mode and the low-power mode. In the sleep mode, the IO may be in an indeterminate state. At this time, the IO can be controlled by the low-power control unit to avoid leakage current caused by the indeterminate state.

4.1.2.1. Input

The following example shows how to configure PB0 into an input function under the control of a low-power control unit:

```
/* config PB0 IO mux */
pmu_set_pin_mux(GPIO_PORT_B, GPIO_BIT_0, PMU_PIN_FUNC_GPIO);
/* set PB0 to input mode */
pmu_set_pin_dir(GPIO_PORT_B, (1<<GPIO_BIT_0), GPIO_DIR_IN);
/* get PORTB value */
pmu_portb_read();
```

4.1.2.2. Output

The following example shows how to configure PB0 into an output function under the control of a low-power control unit:

```
/* config PB0 IO mux */
pmu_set_pin_mux(GPIO_PORT_B, GPIO_BIT_0, PMU_PIN_FUNC_GPIO);
/* set PB0 to input mode */
pmu_set_pin_dir(GPIO_PORT_B, (1<<GPIO_BIT_0), GPIO_DIR_OUT);
/* set PORTB value */
pmu_portb_write(pmu_portb_read() | 0x01);
```

4.1.2.3. Pull Up and Down

The following example shows how to configure pull-up/pull-down resistors and cancel the configuration:

```
/* set PA2 pull down */
pmu_set_pin_pull(GPIO_PORT_A, (1<<GPIO_BIT_2), GPIO_PULL_DOWN);
/* set PA3 pull up */
pmu_set_pin_pull(GPIO_PORT_A, (1<<GPIO_BIT_3), GPIO_PULL_UP);
/* remove PA4 pull down and pull up */
pmu_set_pin_pull(GPIO_PORT_A, (1<<GPIO_BIT_4), GPIO_PULL_NONE);
```

4.1.2.4. GPIO Level Monitoring

There is a GPIO level monitoring module in the low - power control unit. Its implementation mechanism is similar to the XOR operation and is used to monitor the level changes of the pins. When the level of the monitored pin is inconsistent with the set initial value, a PMU interrupt will be generated. The PMU interrupt can wake the system from sleep, so the wake - up function of the GPIO is implemented in this way.

The SDK provides APIs for configuring the monitored objects:

```
void pmu_port_wakeup_func_set(enum system_port_t port, uint8_t bits)
{
    uint8_t last_status_reg = PMU_REG_PORTA_LAST_STATUS;

    last_status_reg += port;

    /* PMU control */
    pmu_set_pin_to_PMU(port, bits);

    /* Config input, Pull up */
    pmu_set_pin_pull(port, bits, GPIO_PULL_UP);
    pmu_set_pin_dir(port, bits, GPIO_DIR_IN);

    /* Read the current value and write */
    ool_write(last_status_reg, ool_read(last_status_reg));

    /* XOR Enable */
    pmu_set_pin_xor_en(port, bits, true);

    /* XOR interrupt enable */
    pmu_enable_isr(PMU_GPIO_XOR_INT_EN);
}
```

Taking enabling PD0 and PD1 as an example, the following demonstrates how to call the above functions to implement monitoring:

```
pmu_port_wakeup_func_set(GPIO_PORT_D, (1<<GPIO_BIT_0) | (1<<GPIO_BIT_1));
```

When the PD0 or PD1 level changes, a PMU interrupt is generated:

```
__attribute__((section("ram_code"))) void pmu_isr(void)
{
    uint16_t state = pmu_get_isr_state();
    pmu_clear_isr_state(state);
    if(state & PMU_GPIO_XOR_INT_STATUS) {
        uint32_t port_value = ool_read32(PMU_REG_PORTA_LAST_STATUS);
        LOG_INFO(NULL, "current port state is 0x%08x\r\n", port_value);
        ool_write32(PMU_REG_PORTA_LAST_STATUS, port_value);
    }
}
```

4.1.3. GPIO Control (New Driving Method)

In the FR8000 SDK, the driver code `driver_gpio.c` has been newly added. The old - version GPIO driver is still retained, and the newly added GPIO driver is more flexible and simpler.

Taking output as an example:

```
GPIO_Handle.Pin      = GPIO_PIN_0|GPIO_PIN_1;
GPIO_Handle.Mode     = GPIO_MODE_OUTPUT_PP;
gpio_init(GPIO_B, &GPIO_Handle);
gpio_write_pin(GPIO_B, GPIO_PIN_0|GPIO_PIN_1, GPIO_PIN_SET);
gpio_write_pin(GPIO_B, GPIO_PIN_0|GPIO_PIN_1, GPIO_PIN_CLEAR);
gpio_write_group(GPIO_B, 0x03);
gpio_write_group(GPIO_B, 0x00);
```

For initialization, you need to select the PINs. You can input 8 PINs at a time and select the GPIO function. Then call the initialization function. For output, you can control single or multiple PINs, and you can also control an entire group. Here, two functions are provided for users to use.

Taking input as an example:

```
GPIO_Handle.Pin      = GPIO_PIN_2|GPIO_PIN_3;
GPIO_Handle.Mode     = GPIO_MODE_INPUT;
GPIO_Handle.Pull     = GPIO_PULLDOWN;
gpio_init(GPIO_B, &GPIO_Handle);
gpio_read_group(GPIO_B);
gpio_read_pin(GPIO_B, GPIO_PIN_2);
gpio_read_pin(GPIO_B, GPIO_PIN_3);
```

For initialization, you need to select the PINs. You can input 8 PINs at a time, select the GPIO function, and configure the pull - up or pull - down resistors. Then call the initialization function. When reading the PIN states, you can read the state of a single PIN or the states of an entire group.

Taking EXTI (External Interrupt) as an example:

```
GPIO_Handle.Pin      = GPIO_PIN_0/GPIO_PIN_1;
GPIO_Handle.Mode     = GPIO_MODE_EXTI_IT_RISING;
GPIO_Handle.Pull     = GPIO_PULLDOWN;
gpio_init(GPIO_B, &GPIO_Handle);
while (1)
{
    if (exti_get_LineStatus(8))
    {
        exti_clear_LineStatus(8);
    }
    if (exti_get_LineStatus(9))
    {
        exti_clear_LineStatus(9);
    }
}
```

For initialization, you need to select the PINs. You can input 8 PINs at a time and select the GPIO function. Here, you can choose to trigger the interrupt on the rising edge, falling edge, high - level, or low - level, and configure the pull - up or pull - down resistors. Then call the initialization function. You can use either the interrupt method or the loop query method to read the state of the EXTI line.

Taking the multiplexed peripheral as an example:

```
GPIO_Handle.Pin      = GPIO_PIN_0/GPIO_PIN_1;
GPIO_Handle.Mode     = GPIO_MODE_AF_PP;
GPIO_Handle.Pull     = GPIO_PULLDOWN;
GOIO_Handle.Alternate = GPIO_FUNCTION_4;
gpio_init(GPIO_B, &GOIO_Handle);
```

When using peripheral functions such as UART, SPI, and I2C, you need to configure the GPIO as a multiplexed function. Users can refer to Appendix I of the FR8000 chip manual to determine the multiplexed functions.

4.2. ADC

The ADC driver provides functions such as initialization, selecting ADC channels, measuring the internal temperature, measuring the battery voltage, obtaining conversion results, enabling, querying, and clearing interrupts.

Taking the ADC channel as an example:

```
ADC_InitParam.ADC_CLK_DIV      = 5;
ADC_InitParam.ADC_SetupDelay    = 80;
ADC_InitParam.ADC_Reference     = ADC_REF_LDO10;
ADC_InitParam.FIFO_Enable       = FIFO_ENABLE;
ADC_InitParam.FIFO_AlmostFullLevel = 8;
adc_init(ADC_InitParam);
adc_Channel_ConvertConfig(ADC_CHANNEL_0|ADC_CHANNEL_1|ADC_CHANNEL_2|ADC_CHANNEL_3);
adc_convert_enable();
```

The initialization parameters from top to bottom are as follows: ADC sampling clock division factor, channel acquisition setup time delay, reference voltage, FIFO enable control, and FIFO nearly full threshold. If the FIFO is enabled, the result value can only be obtained by reading the ADC's DATA register through the `adc_get_data()` function. Additionally, when measuring the internal temperature and battery voltage, the result value can also only be obtained via the `adc_get_data()` function.

When the FIFO is not used, the result values of channel conversion can be obtained by reading the ADC's DATA0 - DATA7 registers through the `adc_get_channel_data()` function.

When only a single channel needs to be measured, it is recommended to enable the FIFO. If multiple channels need to be measured, do not enable the FIFO.

Note: Channel conversion and the measurement of internal signals such as temperature and battery voltage cannot be performed simultaneously.

4.3. DMA

The usage scenarios of DMA are mostly for data transfer from memory to peripherals or from peripherals to memory. Here, we take the I2S transmission as an example to refer to the DMA configuration and usage.

When the target or address is a peripheral, users need to manually assign the DMA request number. The SDK has already encapsulated this functionality. In the `driver_system.h` file, you can see the peripherals available for configuration, as shown below:

```
#define __DMA_REQ_ID_UART0_RX(__REQ_ID__)
#define __DMA_REQ_ID_UART0_TX(__REQ_ID__)
#define __DMA_REQ_ID_SPIO_MASTER_RX(__REQ_ID__)
#define __DMA_REQ_ID_SPIO_MASTER_TX(__REQ_ID__)
#define __DMA_REQ_ID_I2C1_RX(__REQ_ID__)
#define __DMA_REQ_ID_I2C1_TX(__REQ_ID__)
```

Taking I2S transmission as an example

```
__DMA_REQ_ID_I2S_TX(1);

DMA_Chan0.Channel      = DMA_Channel0;
DMA_Chan0.Init.Data_Flow = DMA_M2P_DMAC;
DMA_Chan0.Init.Request_ID = 1;
DMA_Chan0.Init.Source_Inc = DMA_ADDR_INC_INC;
DMA_Chan0.Init.Desination_Inc = DMA_ADDR_INC_NO_CHANGE;
DMA_Chan0.Init.Source_Width = DMA_TRANSFER_WIDTH_32;
DMA_Chan0.Init.Desination_Width = DMA_TRANSFER_WIDTH_32;
dma_init(&DMA_Chan0);
dma_start(&DMA_Chan0, (uint32_t)TxBuffer32, (uint32_t)&I2S->DATA, 256, DMA_BURST_LEN_4,
DMA_BURST_LEN_4);
while(!dma_get_tfr_Status(DMA_Channel0));
dma_clear_tfr_Status(DMA_Channel0);
```

Manually assign the request number of I2S_TX to '1'.

Configure the DMA parameters. From top to bottom, they are: the channel to be used, the transfer direction, the request number, whether the source address is incremented, whether the destination address is incremented, and the bit - width of the transferred data. After configuring the parameters, call the `dma_init()` function to complete the channel

initialization.

Call `dma_start()` or `dma_start_IT()` to enable the data transfer. There is a parameter `DestBurstLen` in the function call. The meaning of this parameter is how many data are transferred at one time. For example, if the data bit - widths `Source_Width` and `Desination_Width` are configured as `DMA_TRANSFER_WIDTH_32`. If `DMA_BURST_LEN_4` is selected, it means that four data of `Source_Width` bit - width are transferred at one time, that is, $4 * 4 = 16$ bytes. When using `BurstLength`, it is necessary to consider whether the destination address or source address has a FIFO space to accommodate multiple bytes of data at one time. The total amount of data to be transferred is determined by the parameter `Size` (the unit of `Size` is bytes).

Finally, the query method or interrupt method can be used to determine whether the DMA transfer is completed.

In addition, DMA also supports linked - list transmission. Each linked - list contains the following information for each transmission: 1. The destination address; 2. The source address; 3. The base address of the linked - list for the next transmission (if this is the last transmission, assign it to `NULL`); 4. The control register. As shown below:

```
typedef struct DMA_NextLink
{
    uint32_t SrcAddr;           /* source address */
    uint32_t DstAddr;           /* desination address */
    struct DMA_NextLink *Next;  /* Next Link */
    REG_CTL1_t CTL1;            /* Control Register for Channel */
    REG_CTL2_t CTL2;            /* Control Register for Channel */
}DMA_LLI_InitTypeDef;
```

After the first linked list transmission is completed, the value of the `*Next` of the first linked list will be queried. If it is not `NULL`, the transmission of the next linked list will be carried out.....

4.4. I2C

The I2C driver provides functions such as initialization, blocking data transmission and reception for the master, interrupt - based data transmission and reception for the master, and EEPROM read - write operations.

Take reading and writing EEPROM as an example:

```
I2C1_Handle.I2Cx = I2C1;  
I2C1_Handle.Init.I2C_Mode = I2C_MODE_MASTER_7BIT;  
I2C1_Handle.Init.SCL_HCNT = 500;  
I2C1_Handle.Init.SCL_LCNT = 500;  
I2C1_Handle.Init.Slave_Address = 0xAE;  
i2c_init(&I2C1_Handle);  
NVIC_EnableIRQ(I2C1_IRQn);  
// EEPROM write/read  
i2c_memory_write(&I2C1_Handle, 0xA0, 32*0, &TxBufferI2C[32*0], 32);  
i2c_memory_read(&I2C1_Handle, 0xA0, 0, RxBufferI2C, 256);
```

First, initialize the parameters:

- **I2Cx**: Select which I2C to use.
- **I2C_Mode**: Select the master - slave mode and the 7 - bit or 10 - bit address mode.
- **SCL_HCNT**: Configure the high - level holding time of SCL. The unit is the I2C peripheral clock source.
- **SCL_LCNT**: Configure the low - level holding time of SCL. The unit is the I2C peripheral clock source.
- **Slave_Address**: If the slave mode is selected, configure the address of the slave itself.

After configuring the initialization parameters, call the `i2c_init()` function to initialize the I2C. Then you can call the read and write functions.

4.5. Other

For other peripherals, you can refer to the relevant driver programs and the FR8000 specification.

5. System Function

5.1. System Clock

The current system can run at two main frequencies of 48MHz and 96MHz. You can set and get the current system clock using the functions `system_set_clock` and `system_get_clock` respectively. Since the bus clock is consistent with the system clock, after switching the system clock, the clocks of some peripherals and the bus will also change accordingly.

5.2. System Sleep

The SDK provides `system_sleep_enable` and `system_sleep_disable` to enable and disable the sleep function. It should be noted that after calling `system_sleep_enable`, the system will not immediately enter the sleep state. Instead, it enables the monitoring function to determine whether the system can enter sleep. Whether the system can enter sleep is determined by the content described in Section 1.4.3.

5.3. Other

5.3.1. Program Running Time

The SDK provides the function `system_get_curr_time` to get the running time of the system from power - on to the moment of calling. The unit of this function is ms. After the timer reaches 83886079, it will restart counting from 0.

5.3.2. SDK Compilation Time

The user program can get the compilation time of the SDK through the function `get_SDK_compile_date_time` for version tracking.

6. OTA

FR8000 uses a dual - backup method to implement OTA (Over - the - Air) upgrades.

The currently running firmware is stored in Area A, and the firmware to be upgraded is stored in Area B. The bootloader determines the validity of the two firmware storage areas based on the content of the firmware information area.



Figure 6-1 Firmware Information Area Data Format

During system startup, the Bootloader first checks the "Configuration Information Validity Flag" in the "Firmware Information of Area A". If the check passes, it is considered that there is a valid firmware stored in Area A. Then, it checks the "Validity Flag of Area B" in the "Firmware Information of Area B". If the check passes, it is considered that there is a firmware to be upgraded stored in Area B. If the check fails, the Bootloader will run the firmware in Area A, which has passed the check. When a firmware to be upgraded is detected, the Bootloader will upgrade the firmware from Area B to Area A.

6.1. Firmware Generation

The bin file generated by project compilation can be directly used by PC programming tools and mass - production programming tools. Its basic format is as follows:

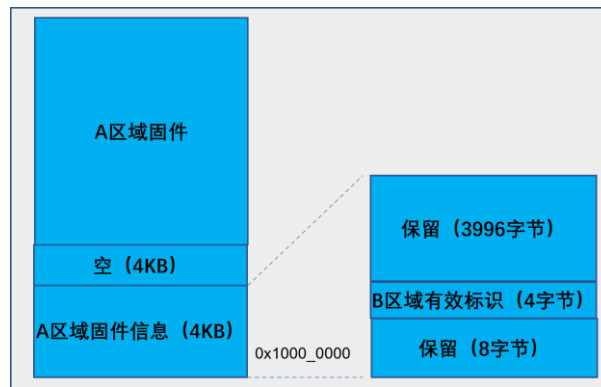


Figure 6-2 The basic format of the compiled bin file is as follows

If this file is used for OTA upgrade, it is necessary to pad the end of the firmware with 0xFF so that the total size of the firmware is aligned to 4KB. Then, add some content to the firmware information field. The orange fields should be calculated and filled in according to the actual information, and the green fields should be filled with 0xFFFFFFFF:

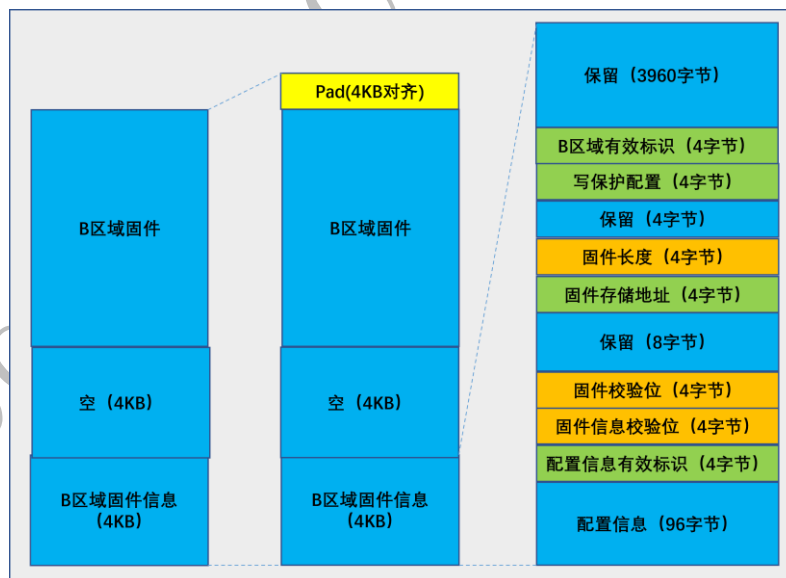


Figure 6-3 Fields to be added to the information of the firmware to be upgraded

6.2. Firmware Writing

Firmware writing is divided into two parts: one is the firmware information area; the other is the firmware area. When performing OTA firmware writing, it is recommended to follow the steps below:

1. Write the firmware information area to the "Firmware Information of Area B";
2. Obtain the end position of the firmware in Area A and align it upward to a multiple of 4KB as the starting position (B_offset) for writing the firmware area.
3. Store the firmware area of the OTA firmware in the FLASH space starting from B_offset.
4. After the storage is completed, the program checks whether the "Firmware Information Check Bit" and the "Configuration Information Check Bit" match the actual values. If the check passes, fill in the "Firmware Storage Address" (B_offset), the "Validity Flag of Area B", and the write - protection configuration. If the check fails, the OTA upgrade fails.
5. If step 4 passes, the system restarts, and the bootloader is responsible for the actual firmware upgrade.

Contact Information

Website: www.freqchip.com

Sales Email: sales@freqchip.com

Phone: +86-21-5027-0080

Feedback:

Freqchip welcomes feedback on this product and this document. If you have comments or suggestions, please send an email to docs@freqchip.com.